



MINISTERIO
DE EDUCACIÓN, CULTURA
Y DEPORTE
Programa financiado por el Ministerio de Educación, Cultura y Deporte
y cofinanciado por el Fondo Social Europeo



UT 1: EXPLOTACIÓN DE SISTEMAS INFORMÁTICOS

1º DAM – Sistemas Informáticos

ECL981725Curso 2025-2026

1.- INTRODUCCIÓN:

***CONCEPTOS BÁSICOS, DEFINICIONES Y EVOLUCIÓN
HISTÓRICA DE LOS SISTEMAS INFORMÁTICOS***

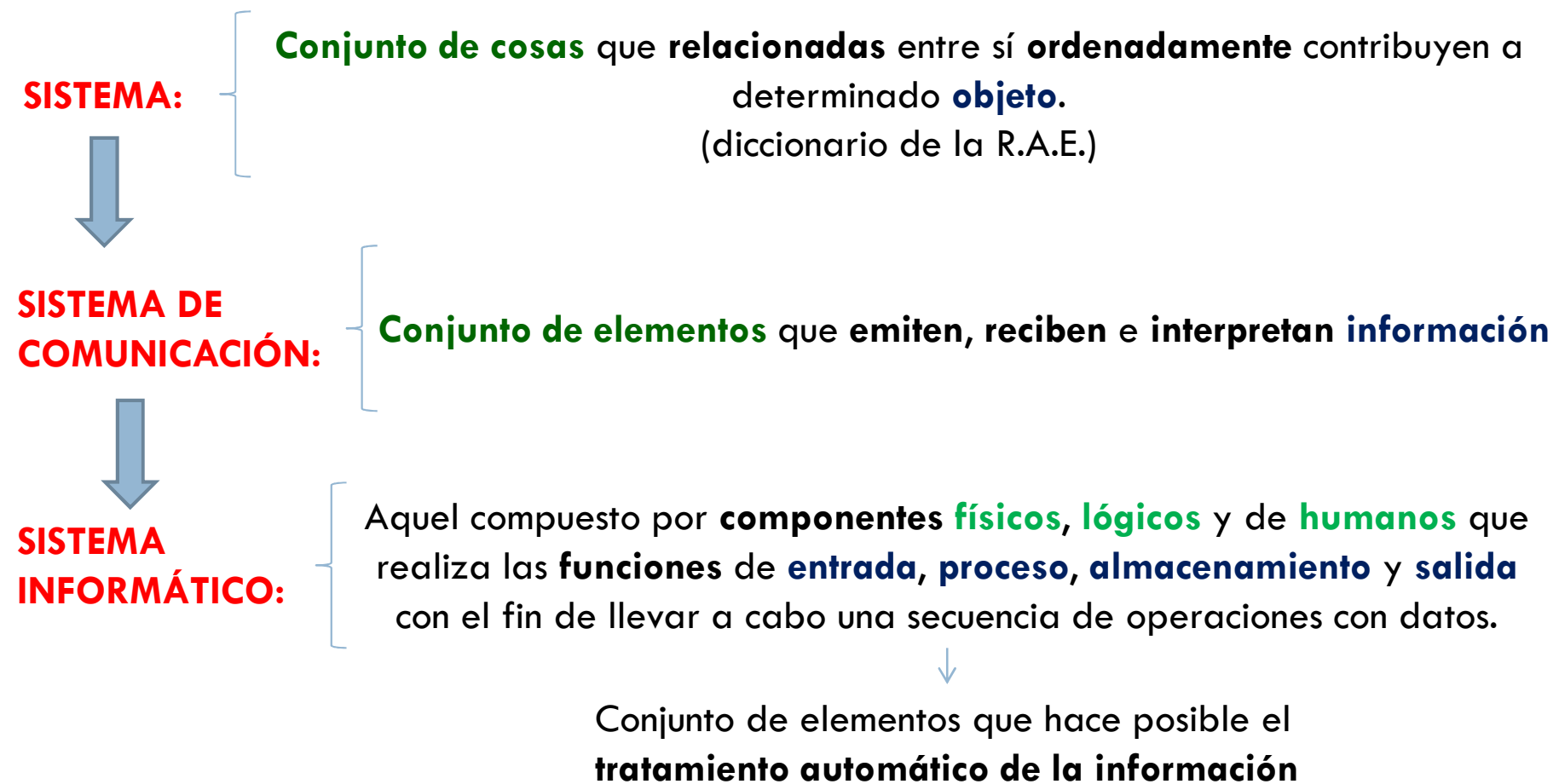
1.1.- CONCEPTOS BÁSICOS INFORMÁTICOS

3



1.1.- CONCEPTOS BÁSICOS INFORMÁTICOS

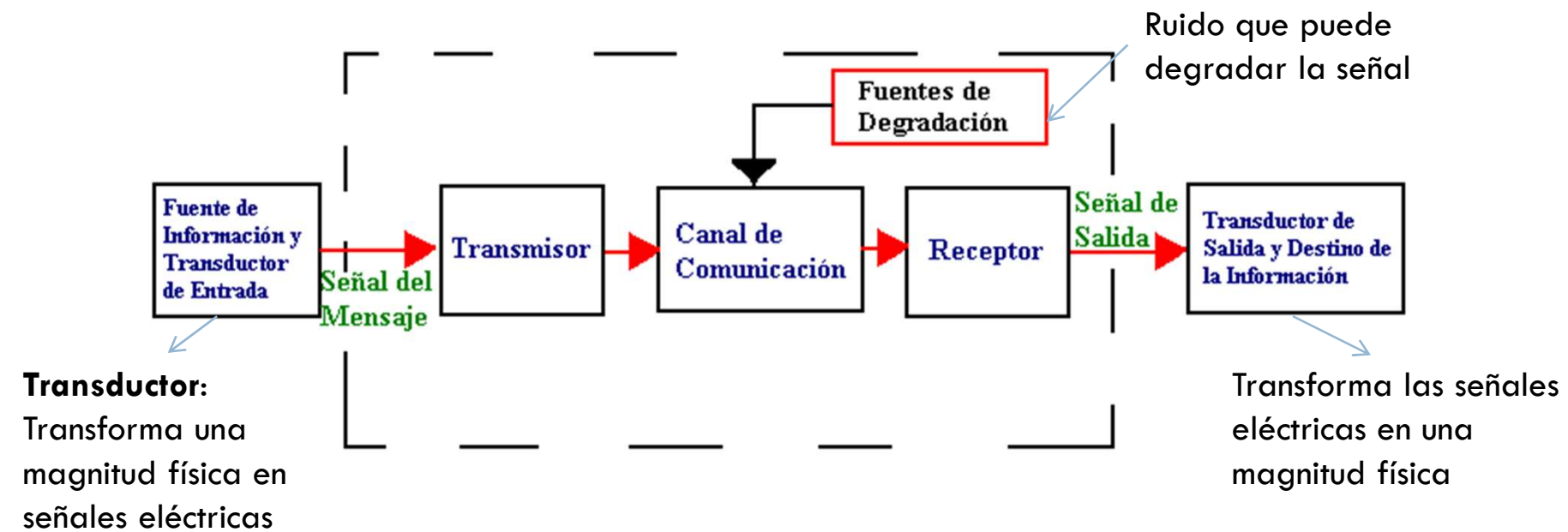
4



1.1.- CONCEPTOS BÁSICOS INFORMÁTICOS

5

- **SISTEMA DE COMUNICACIÓN:**

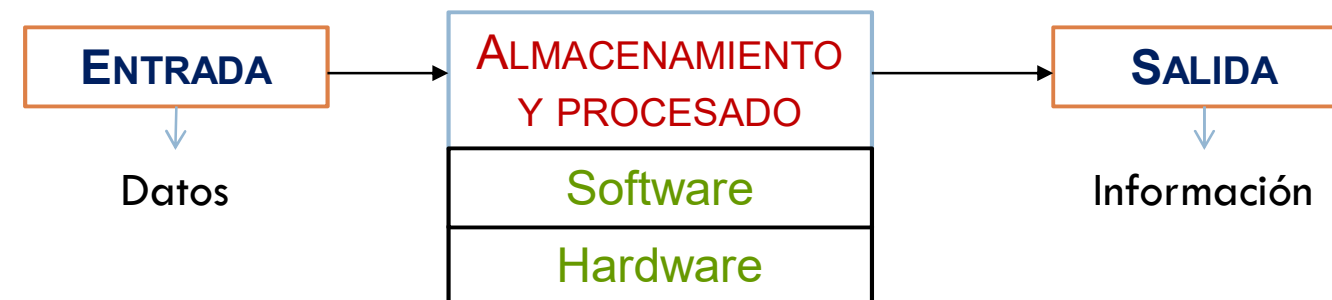


OBJETIVO: → Envío de un **mensaje** (información) desde un sistema **emisor** a otro **receptor** por medio de un **canal de comunicación**

1.1.- CONCEPTOS BÁSICOS INFORMÁTICOS

6

- **SISTEMA INFORMÁTICO:**

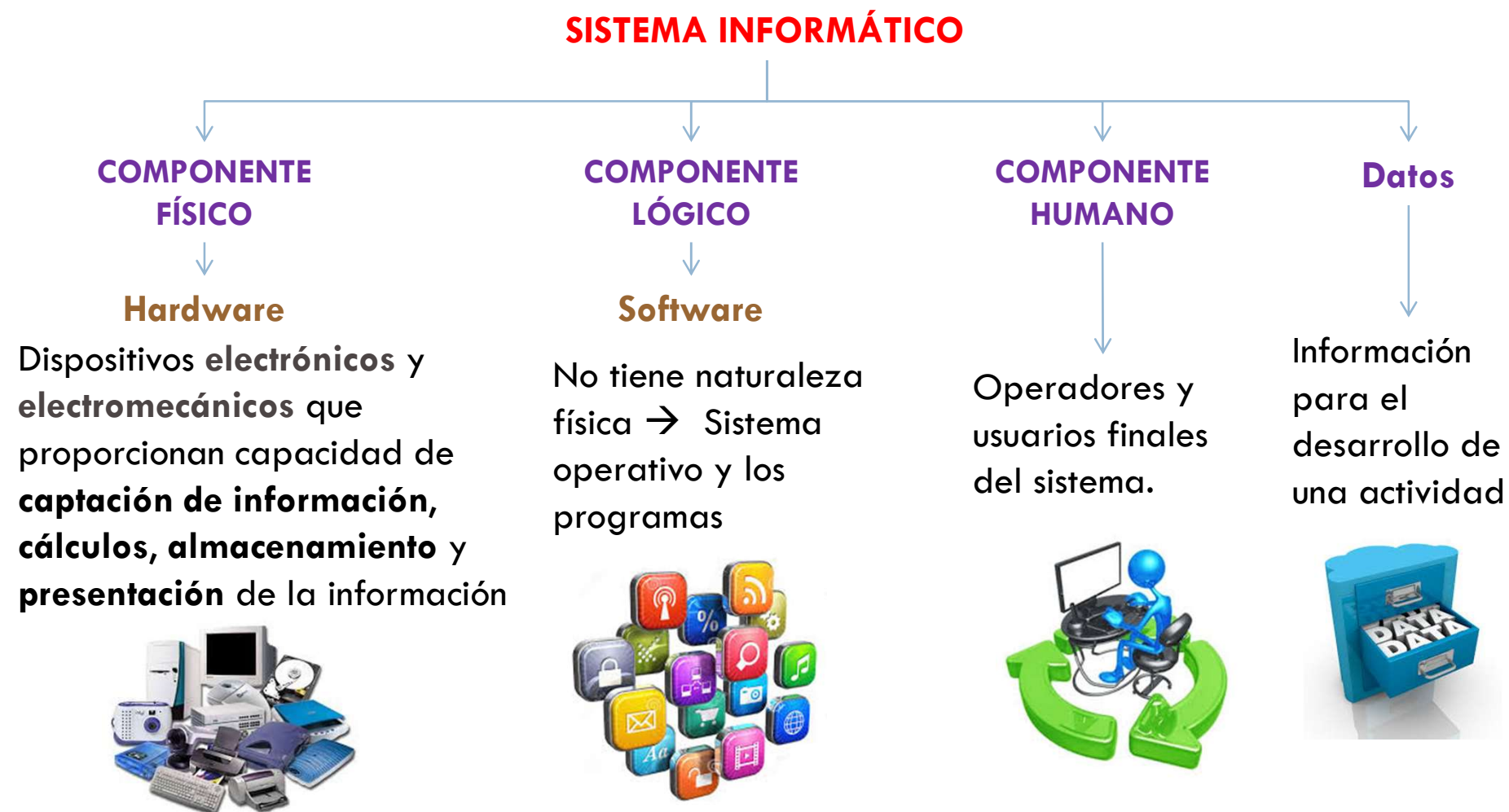


FUNCIONES BÁSICAS:

- **Entrada** de la información
- **Procesamiento** de la información (puede también realizarse un simple **almacenamiento**)
- **Salida** de la información ya **procesada**

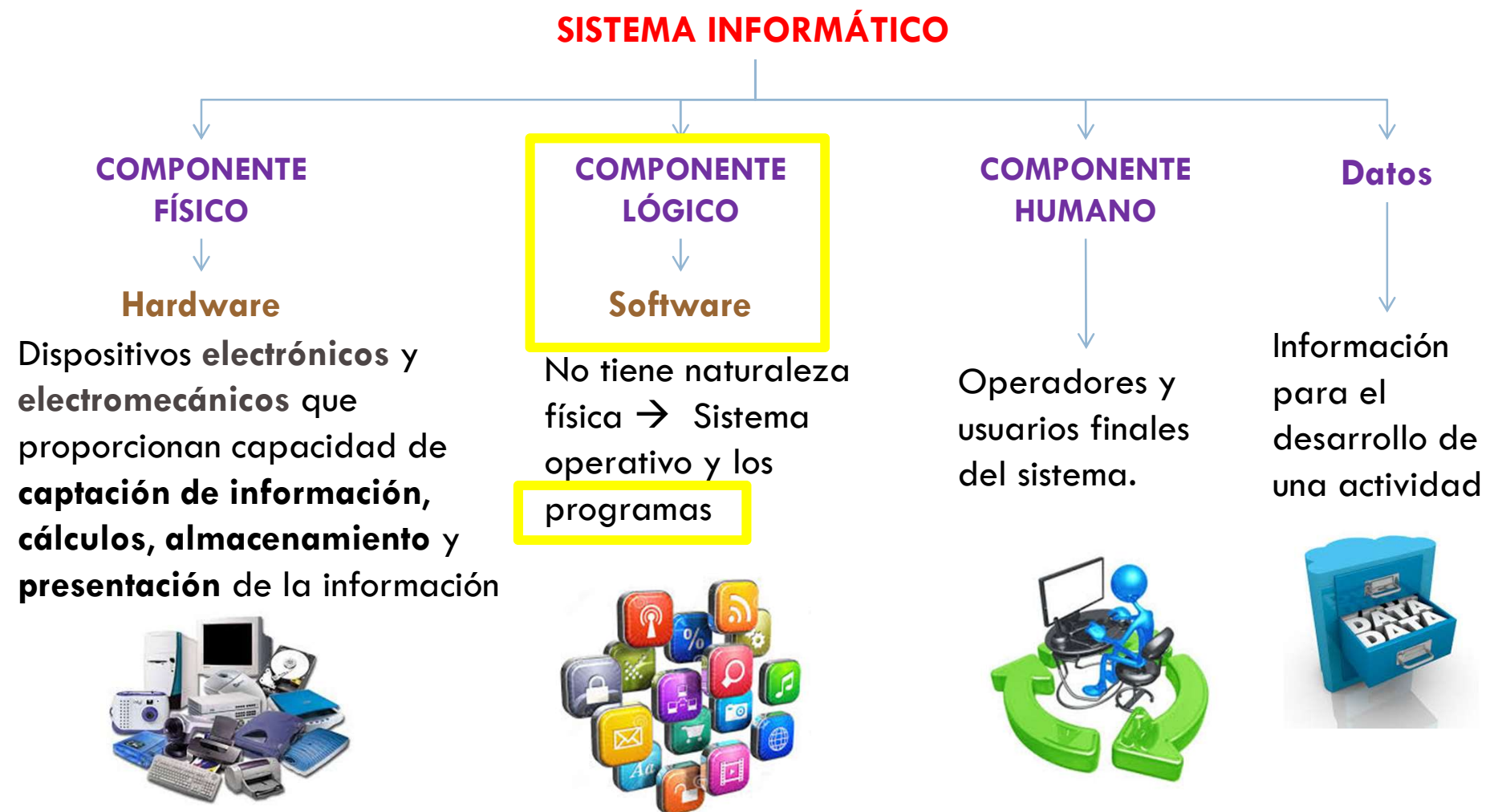
1.2.- COMPONENTES DE UN SISTEMA INFORMÁTICO

7



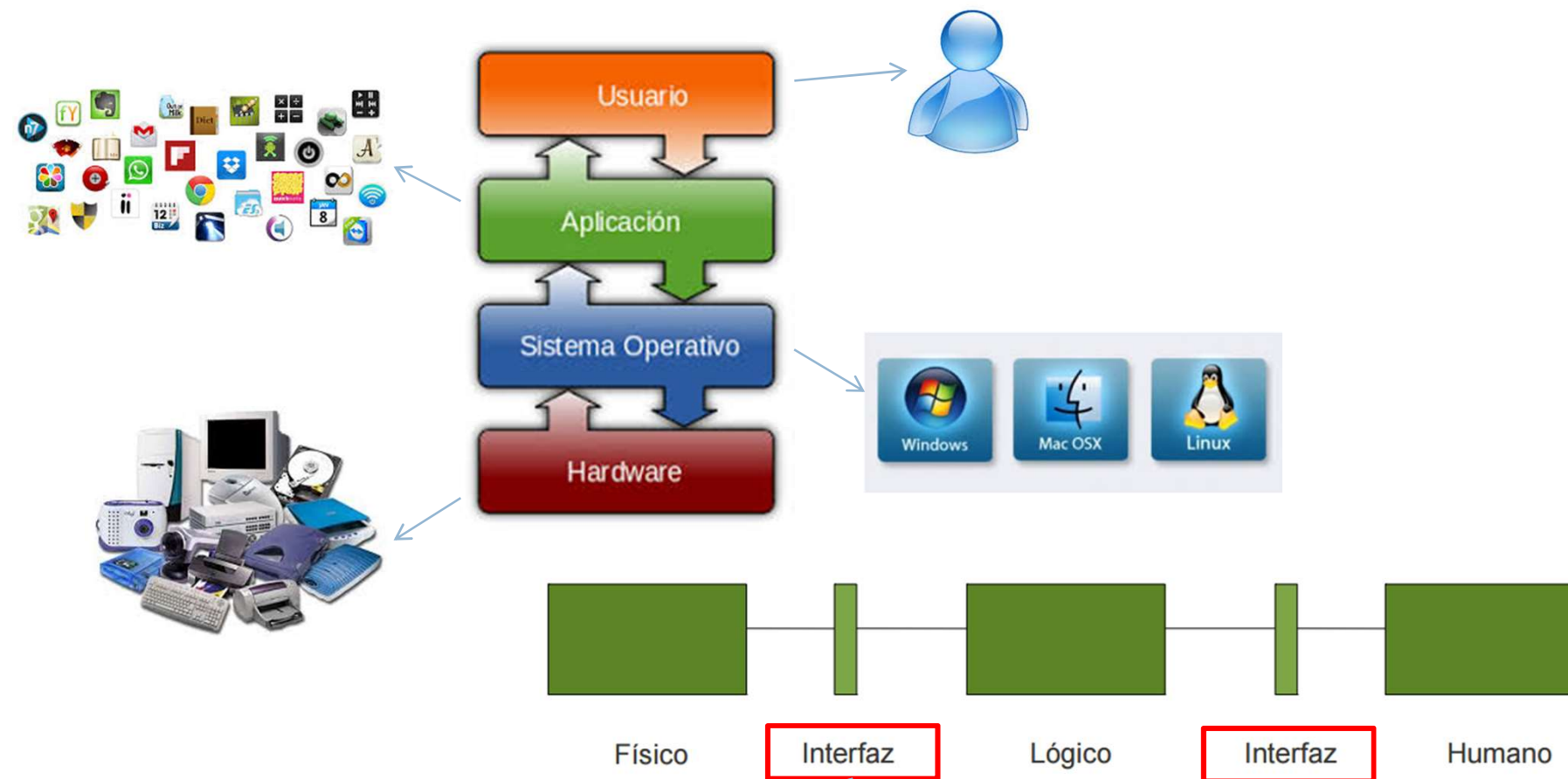
1.2.- COMPONENTES DE UN SISTEMA INFORMÁTICO

8



1.2.- COMPONENTES DE UN SISTEMA INFORMÁTICO

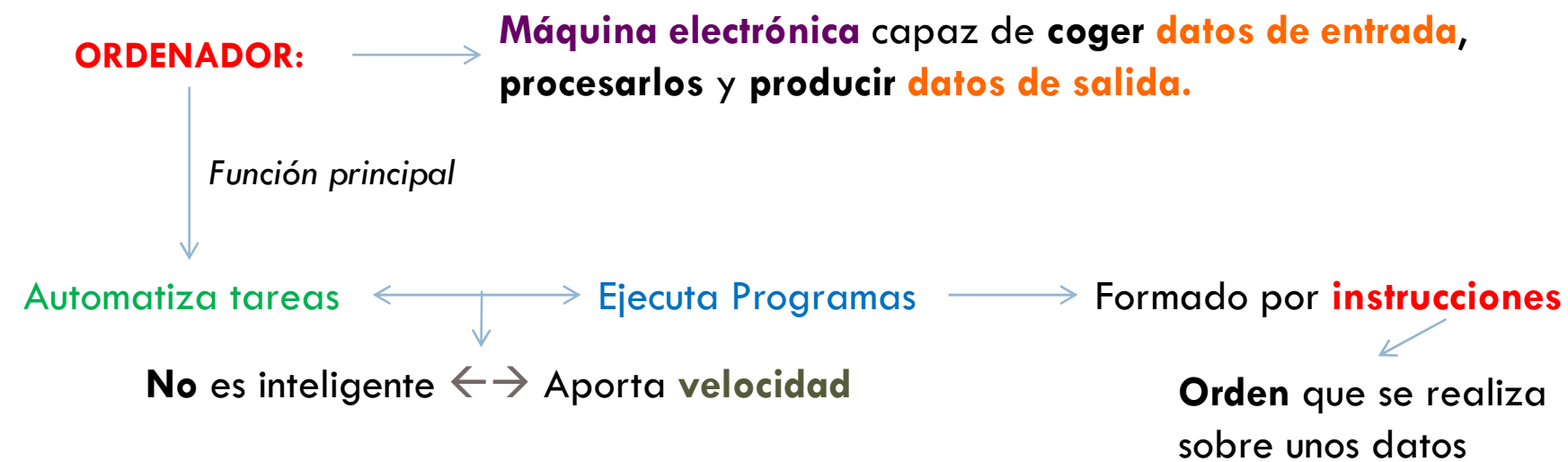
9



Conexión física y funcional entre dos aparatos o sistemas independientes

1.3.- EL ORDENADOR

10



TODAS las tareas que realizamos con un ordenador, para el ordenador sólo son un conjunto de instrucciones que ejecuta
Ejemplo: jugar con el ordenador

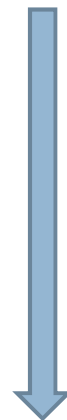


- Instrucción: sumar 3 y 5

1.3.- EL ORDENADOR

11

FUNCIONES BÁSICAS:



COMPONENTES HARDWARE:

Para realizarlas cuentan con...

- 1.- Recibe información (**entrada**)
- 2.- Procesa la información recibida
- 3.- Almacena la información
- 4.- Produce información (**salida**)

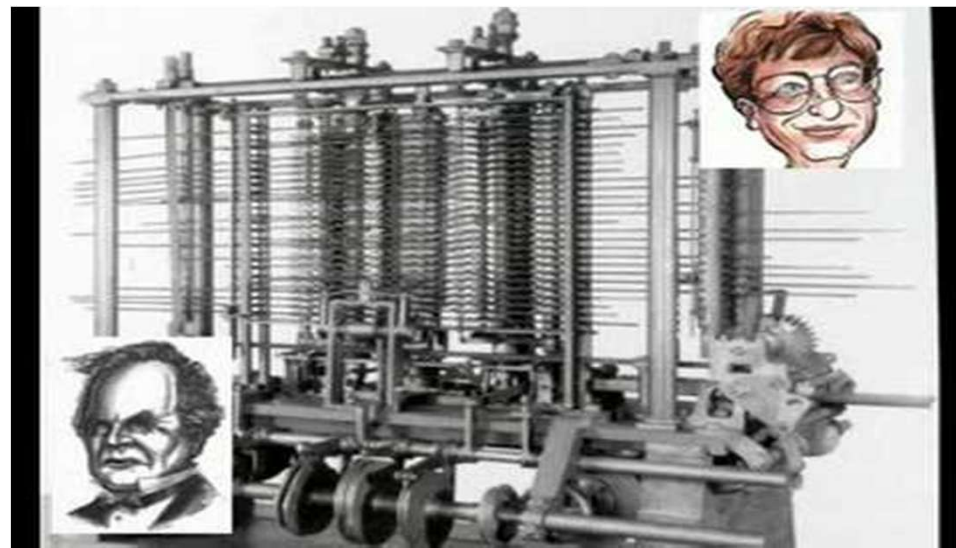
- Dispositivos de entrada
- Dispositivos de salida
- Unidad Central de Proceso (CPU)
- Memoria principal
- Dispositivos de almacenamiento secundario

Hardware

VS

Software

EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS



1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

13

Preinformática (antes de 1940):

Ábaco (???)

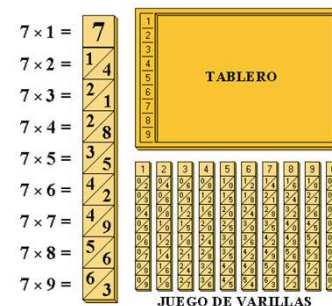


Instrumento que sirve para **efectuar operaciones aritméticas sencillas** (sumas, restas y multiplicaciones)

Considerado el precursor de la calculadora digital moderna

Funcionamiento

Tablas logarítmicas
(John Napier) (1614)



Tablero con reborde en el que se colocarán las varillas neperianas (tiras de madera, metal o cartón grueso) para realizar las operaciones de **multiplicación** o **división**. El tablero tiene su reborde izquierdo dividido en 9 casillas en las que se escriben los números 1 a 9.

1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

14

Pascalina
(*Blaise Pascal*) (1642)



→ Primera **calculadora** que funcionaba a base de
ruedas y engranajes

Cuando una rueda giraba completamente sobre su eje, hacía
avanzar un grado a la siguiente.
Las ruedas representaban el «**sistema decimal de numeración**»

**Máquina diferencial y
máquina analítica**
(*Charles Babbage*) (S XIX)



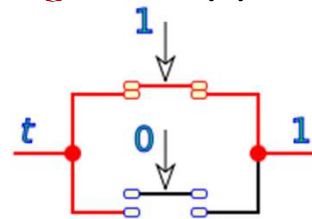
→ Calculadora mecánica de propósito especial,
diseñada para calcular **funciones polinómicas**.

Máquina diferencial Charles Babbage

1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

15

Algebra de boole
(George Boole) (1847).

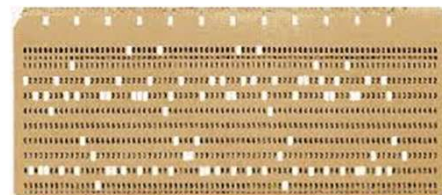


Permite definir el modo de realizar
cálculos y comparaciones en binario

Estructura algebraica que esquematiza las operaciones lógicas **Y, O, NO** y **SI (AND, OR, NOT, IF)**, así como el conjunto de operaciones **unión, intersección y complemento**.

Máquina tabuladora de Hollerith
(Herman Hollerith) (1890)

Permite crear censos con **tarjetas perforadas**



Anécdota...

Se tardaron sólo 3 años en perforar unas 56 millones de tarjetas

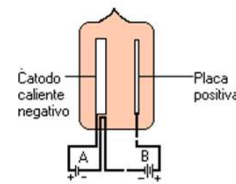
1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

16

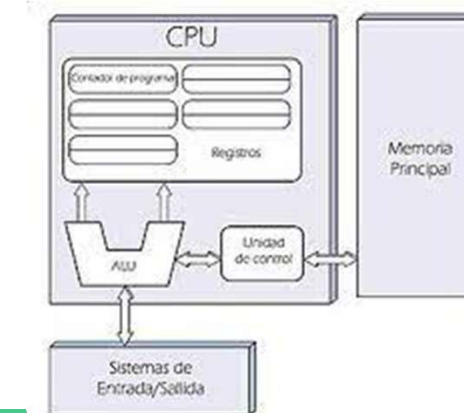
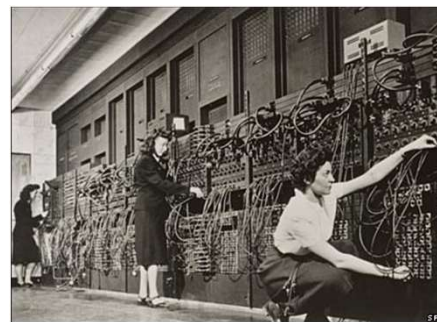
1º GENERACIÓN (1940-1950):

Válvulas de vacío

II Guerra Mundial → Fines militares y científicos



- Máquinas **grandes** y requerían programación “**física**”.
- Poco fiables
- Potencia de cálculo → **menos** que una calculadora de bolsillo
- Von Neumann establece los principios de su arquitectura en el borrador de la **máquina EDVAC** que no llegaría a construirse
- Máquina **ENIAC**



1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

17

2º GENERACIÓN (1950-1965):

- Transistor** →
- Máquinas más pequeñas, menor consumo
 - **ATLAS** → Primer computador de transistores

Primeros periféricos

- Tarjetas perforadas
- Cinta magnética
- Impresora



- Supercomputadoras** → Máquinas con capacidades de cálculo muy superiores a las computadoras comunes y de escritorio y que son usadas con fines específicos

- Trabajo por lotes** ➤ Ejecución del programa no precisa ningún tipo de interacción con el usuario

Serie IBM 7090

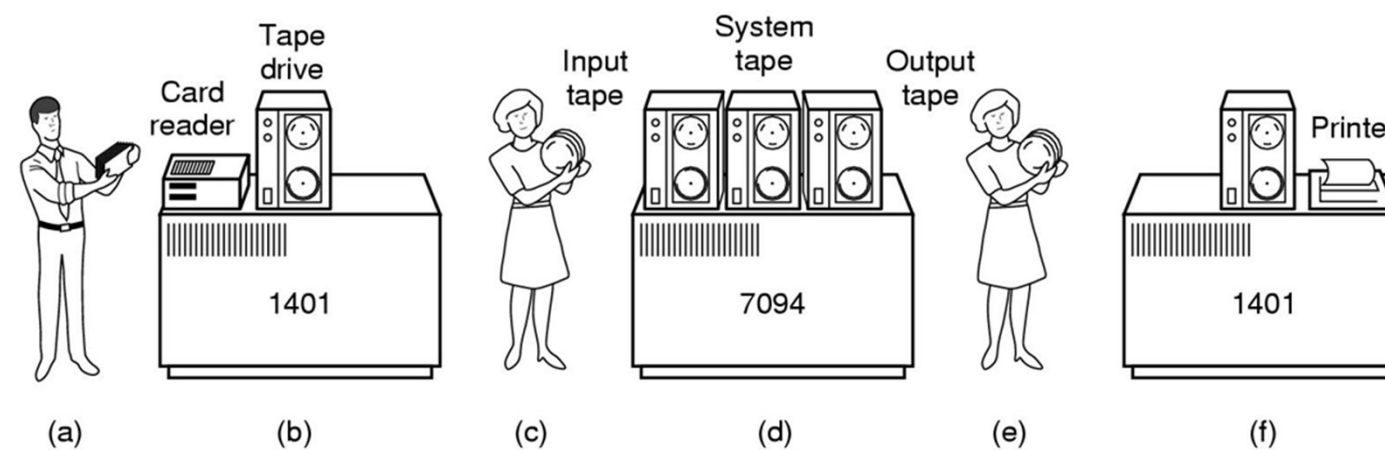
- Una habitación
- 2.900.000 \$ en el mercado

Consola de manejo del IBM 7090 en la NASA



1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

18



Carga las tarjetas perforadas en 1401

El proceso de lectura de las tarjetas perforadas es muy lento

Copia las tarjetas a la cinta magnética

Carga la cinta en 7094, que realiza los cálculos

- El proceso de carga de la cinta es mucho más rápido
- Los cálculos pueden ser complejos y llevar tiempo
- En una cinta se pueden grabar varios programas, de forma que se fueran ejecutando uno tras otro sin necesidad de intervención del operador
- La cinta de sistema contiene un primitivo sistema operativo, llamado cargador, que carga los trabajos que forman el lote
- Mientras se está procesando este lote de trabajos, otro usuario puede estar cargando sus tarjetas en el 1401

Carga la cinta en 1401, que imprime el resultado

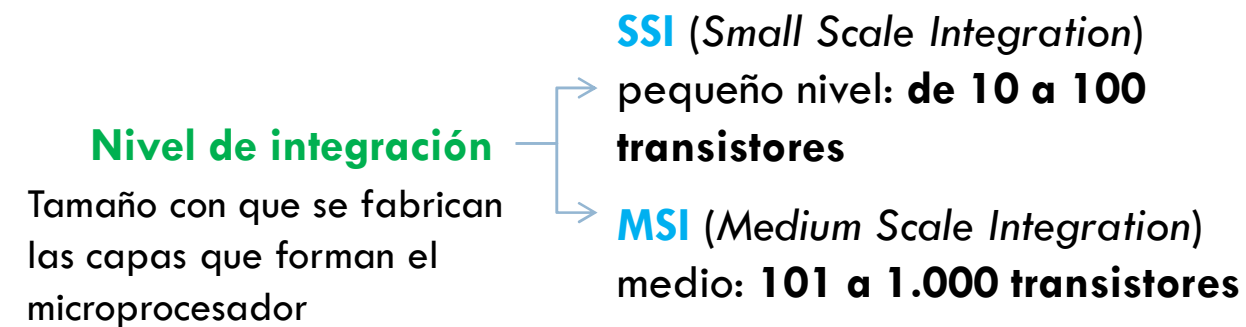
Mientras se imprime, se pueden estar realizando los cálculos de un trabajo posterior

1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

19

3° GENERACIÓN (1965-1975):

Circuitos integrados → Todos los transistores en un chip



Mainframe ↔ **Miniordenador**

Clase de computadora multiusuario, que se encuentran en el **rango intermedio** del espectro computacional; es decir, entre los grandes sistemas multiusuario (**mainframes**) y los más pequeños sistemas monousuarios (microcomputadoras, computadoras personales, o **PC**, etc.).

1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

20

Estación de trabajo → Ordenador de altas prestaciones destinado para trabajo técnico o científico

Serie 360 de IBM {

- Una habitación
- 2.000.000 \$ en el mercado



Incluía un sistema operativo llamado OS con instrucciones para el manejo de la memoria y el procesador

1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

21

4° GENERACIÓN (1975-1983):

Microchip → Millones de transistores

Nivel de integración → **LSI** (*Large Scale Integration*) grande:
1.001 a 10.000 transistores

Microprocesador → integrando **todo el hardware** de la CPU.

Apple I



IBM PC



1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

22

5° GENERACIÓN (1983-1991):

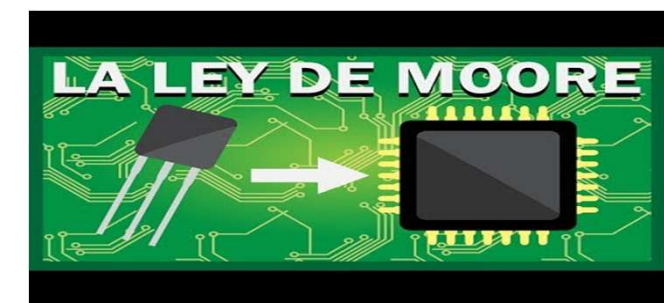
Nivel de integración → **VLSI** (*Very Large Scale Integration*) muy grande:
10.001 a 100.000 transistores

Ley de Moore → Evolución **muy rápida** de la tecnología

El **número de transistores** en un procesador se **duplica** cada
18-24 meses

Con el tiempo:

- Se **reduce** el **tamaño**
- Se **incrementa** la **potencia de cálculo**
- Se **incrementa** su **fiabilidad**
- Se **reduce** el **peso**



1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

23

Bajada de precio de las computadoras, haciéndose accesibles a más personas y ámbitos.

Procesamiento en paralelo → **Muchas instrucciones se ejecutan simultáneamente**, operando sobre el principio de que **problemas grandes**, a menudo se pueden **dividir** en unos más **pequeños**, que luego son resueltos simultáneamente (en paralelo).

Redes de ordenadores



1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

24

6° GENERACIÓN (1992-actualidad):

Nivel de integración → **ULSI** (*Ultra Large Scale Integration*) ultra grande:
100.001 a 1.000.000 transistores
→ **GLSI** (*Extreme Large Scale Integration*) giga grande: **más de 1.000.000 transistores**

Procesadores multinúcleo → Combina **dos o más** microprocesadores **independientes** en un solo paquete, a menudo **un solo circuito integrado**

Empleo masivo de las redes de ordenadores

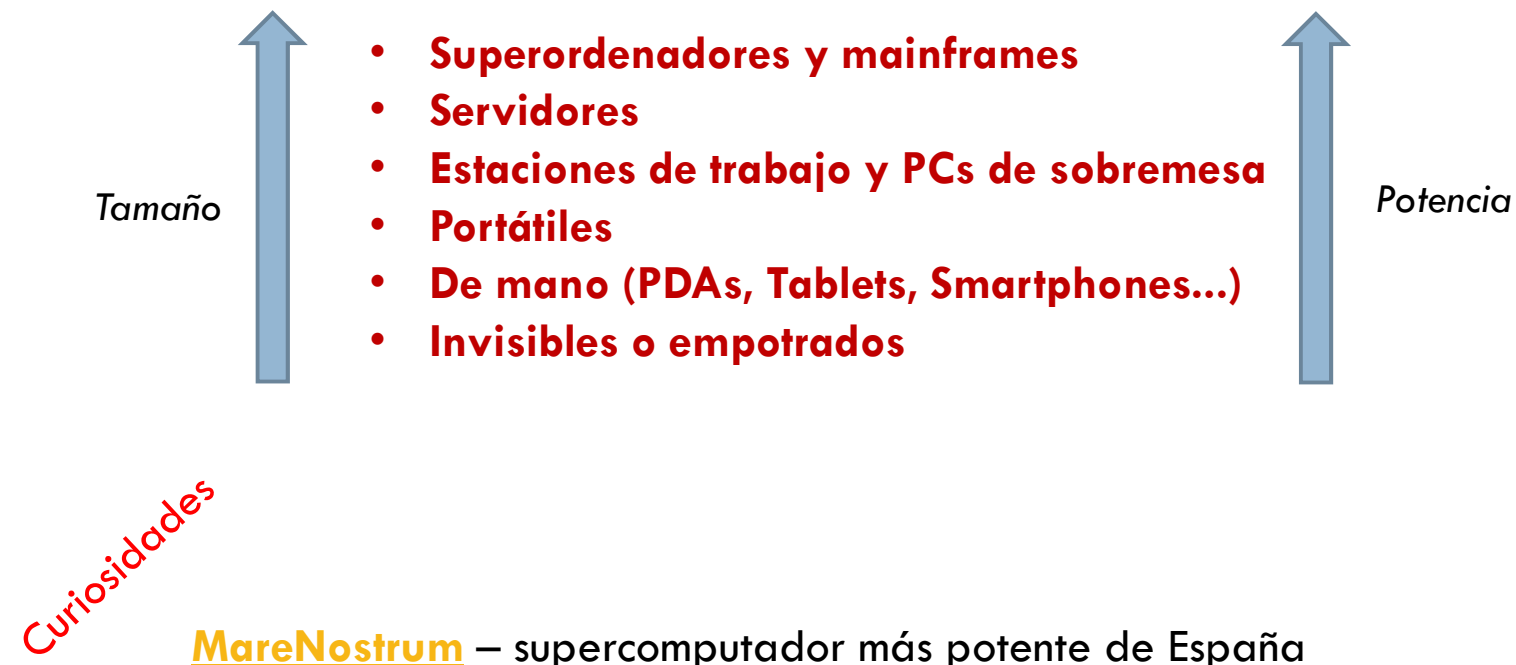
Dispositivos portátiles, Smartphones, tablets...



1.4.- EVOLUCIÓN DE LOS SISTEMAS INFORMÁTICOS

25

Clasificación de los ordenadores basándonos en su tamaño y potencia:

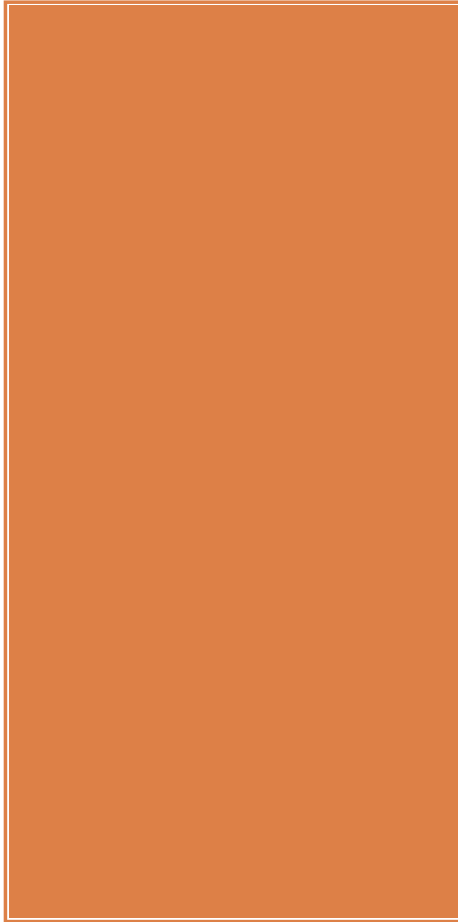




2.- REPRESENTACIÓN DE LA INFORMACIÓN

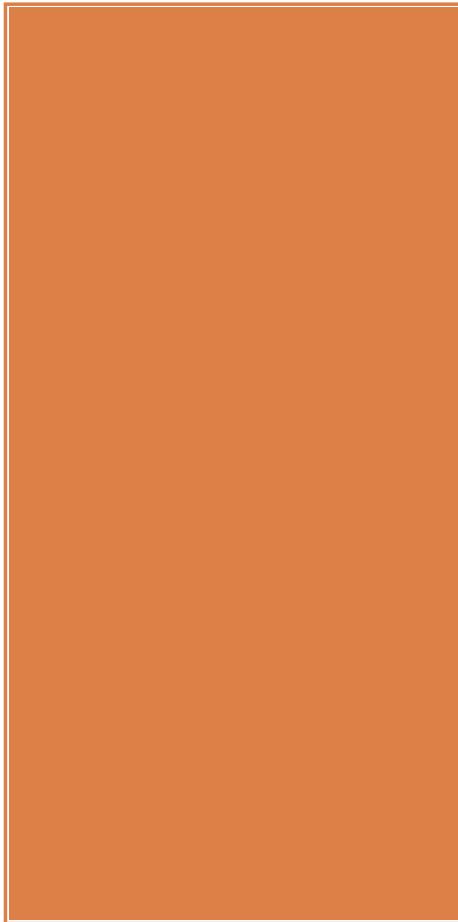
Objetivos

27

- 
- Aprender los métodos de representación numérica de los sistemas: decimal, binario, octal y hexadecimal.
 - Conversión entre los distintos sistemas numéricos.
 - Las operaciones aritméticas en el sistema binario.

Introducción

28

- 
- Los circuitos digitales trabajan con señales bivaluadas: 0 y 1
 - Los computadores se emplean para almacenar todo tipo de información: números enteros, reales, texto, gráficos, audio, video, etc.
 - Esta información ha de reducirse a su representación mediante 0 y 1 ==> CODIFICACIÓN DIGITAL

SISTEMAS DE NUMERACIÓN

29



Los sistemas de numeración son formas de representar cualquier cantidad numérica.

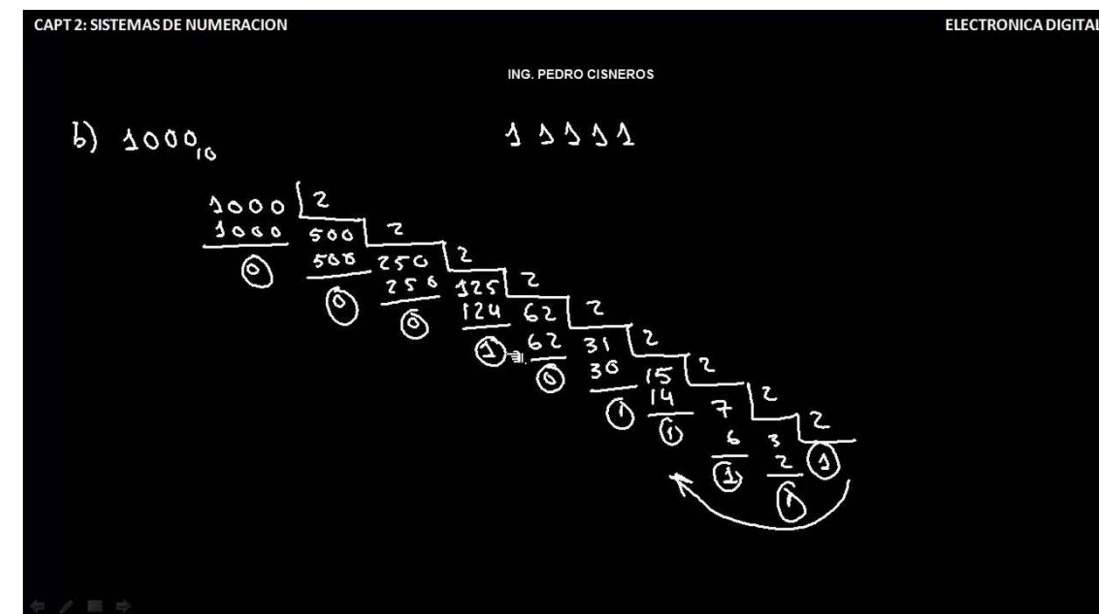


Casi todos los sistemas de numeración son **polinomiales** o **posicionales**.

SISTEMAS DE NUMERACIÓN

□ Características de un sistema polinomial:

- ▣ Todo número es una expresión formada por un conjunto de símbolos, llamados **dígitos**, cada uno con un valor fijo y diferente a los demás.
- ▣ El número de símbolos que se pueden usar en un determinado sistema de numeración constituye su base.
 - Base 10: $\{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$
 - Base 2: $\{ 0, 1 \}$
 - Base 8: $\{ 0, 1, 2, 3, 4, 5, 6, 7 \}$
 - Base 16: $\{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F \}$



SISTEMAS DE
NUMERACIÓN

- El valor numérico que expresa una determinada combinación depende de dos factores: el **valor** de los dígitos y su **posición** en el polinomio.
- Cada posición del dígito tiene un valor intrínseco que aumenta de derecha a izquierda según potencias sucesivas de la base del sistema de numeración empleado.

**Teorema
fundamental de
numeración****Ejemplo:**

- $287_{10} = 2 \cdot 10^2 + 8 \cdot 10^1 + 7 \cdot 10^0$
- $10101_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
- $5735_8 = 5 \cdot 8^3 + 7 \cdot 8^2 + 3 \cdot 8^1 + 5 \cdot 8^0$

En general un número se puede
expresar como:

- $n_m n_{m-1} \dots n_2 n_1 n_0 =$
 - $n_m b^m + n_{m-1} b^{m-1} + \dots + n_2 b^2 + n_1 b^1 + n_0 b^0$

Sistema binario

33



Cuando un sistema tiene base 2 se denomina **sistema binario**. La unidad mínima en sistema binario se denomina bit y puede tener 2 valores 0 o 1.



El sistema de numeración binario tiene muchos usos, desde la programación de microprocesadores, transferencia de datos, cifrado de información, hasta comunicación digital, electrónica y otras áreas relacionadas.



El hecho de trabajar con dos valores hace que se adapte perfectamente a los ordenadores. Cada número en el ordenador es una señal eléctrica.



El sistema binario moderno fue documentado en su totalidad por Leibnitz y más tarde **George Boole** enunció la denominada **Algebra de Boole**, que determina las operaciones básicas sobre números binarios.

El valor de los bits en un número binario es:

$11100101 = 2^7 + 2^6 + 2^5 + 2^2 + 2^0 =$
 $= 128 + 64 + 32 + 4 + 1 = 229$

Valores posicionales en el sistema binario								
Potencia de dos	2⁷	2⁶	2⁵	2⁴	2³	2²	2¹	2⁰
Valor	128	64	32	16	8	4	2	1
Potencia de dos	2¹⁵	2¹⁴	2¹³	2¹²	2¹¹	2¹⁰	2⁹	2⁸
Valor	32768	16384	8192	4096	2048	1024	512	256

Decimal	Binario	Decimal	Binario
0	00000	16	10000
1	00001	17	10001
2	00010	18	10010
3	00011	19	10011
4	00100	20	10100
5	00101	21	10101
6	00110	22	10110
7	00111	23	10111
8	01000	24	11000
9	01001	25	11001
10	01010	26	11010
11	01011	27	11011
12	01100	28	11100
13	01101	29	11101
14	01110	30	11110
15	01111	31	11111

- Tabla de conversion de binario a decimal y viceversa de los 32 primeros valores.

mas octal y hexadecimal



El sistema binario requiere muchos dígitos para almacenar un número



Como alternativa se utilizan los sistemas **octal** (base 8) y **hexadecimal** (base 16).



Estos sistemas tienen mucha relación entre sí:

1 dígito octal = 3 dígitos binarios

1 dígito hexadecimal = 4 dígitos binarios

Sistemas octal y hexadecimal



El sistema octal utiliza la base 8. Utiliza 8 dígitos para representar los datos. Los dígitos son de 0 1 2 3 4 5 6 7



En el sistema hexadecimal se utiliza la base 16. Los dígitos para representar en hexadecimal son 1 2 3 4 5 6 7 8 9 A B C D E F



Estos sistemas tienen mucha relación entre sí:

1 dígito octal = 3 dígitos binarios

1 dígito hexadecimal = 4 dígitos binarios

	Hexadecimal	Octal	Binario
0	0	0	0000
1	1	1	0001
2	2	2	0010
3	3	3	0011
4	4	4	0100
5	5	5	0101
6	6	6	0110
7	7	7	0111
8	8	10	1000
9	9	11	1001
10	A	12	1010
11	B	13	1011
12	C	14	1100
13	D	15	1101
14	E	16	1110
15	F	17	1111

Conversión entre bases

- Se realiza en dos pasos:
 - ▣ Convertir de la base original a base decimal
 - ▣ Convertir de base decimal a la base destino.

Conversión a base decimal

- ⦿ Para convertir un número a base decimal hay que utilizar su representación polinomial.

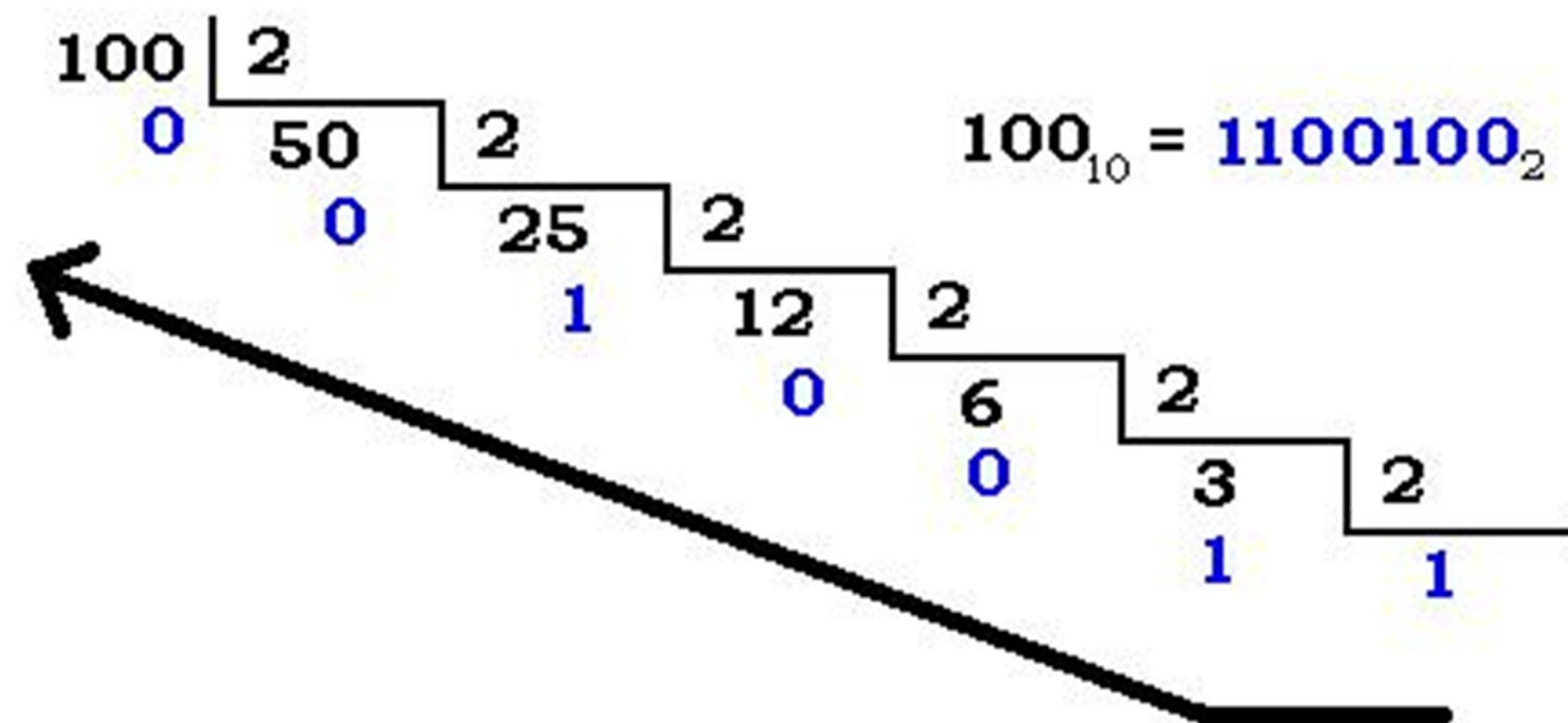
$$\begin{aligned} 12227_8 &= 1*8^4 + 2*8^3 + 2*8^2 + 2*8^1 + 7*8^0 = \\ &= 1*4096 + 2*512 + 2*64 + 2*8 + 7*1 = \\ &= 5271_{10} \end{aligned}$$

Conversión de decimal a binario

40

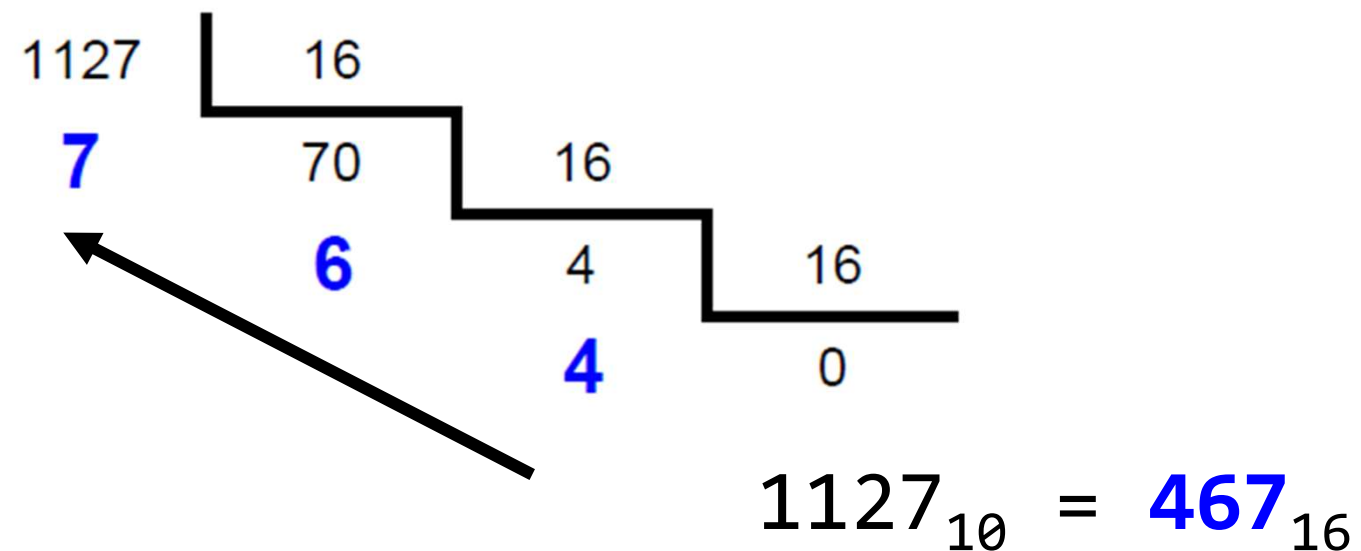
Conversión decimal a binario

Se realizan sucesivas divisiones entre 2 del número y se guardan los restos. Estos restos en orden inverso al obtenido son los dígitos del número en binario



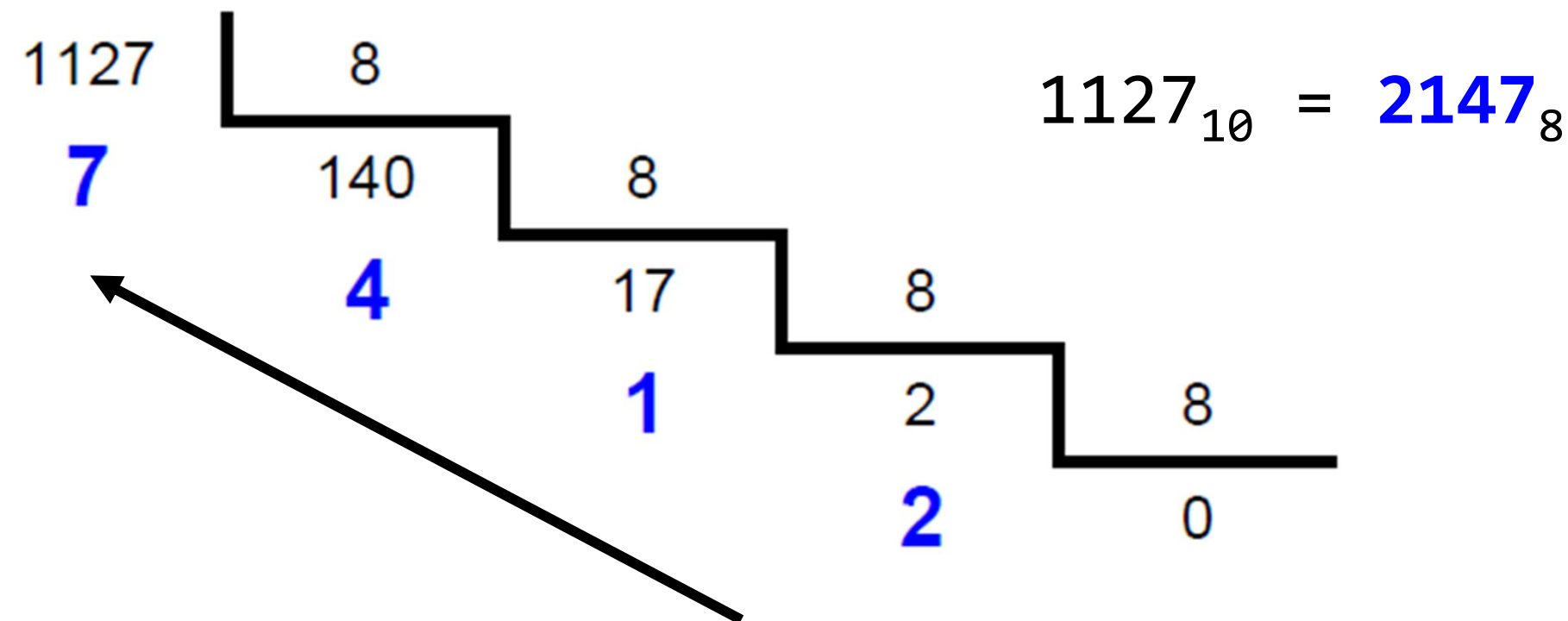
Conversión decimal a hexadecimal y octal

- Se realiza de forma análoga a la conversión a binario pero se divide entre 16 y 8 respectivamente.

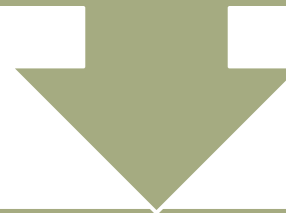


CONVERSIÓN DE DECIMAL A OCTAL

42



**Conversión Binario – Octal
Binario - Hexadecimal**



Otra opción para convertir entre estos sistemas es aprovecharse de la relación entre ellos:

1 dígito octal = 3
dígitos binarios

1 dígito hexadecimal
= 4 dígitos binarios

Hexadecimal a binario

5 3 F A
↓
0101 0011 1111 1010
53FA = 0101001111111010

Octal a binario

5 3 0 1
↓
101 011 000 001
5301 = 101011000001

Binario a hexadecimal

0111 1011 1000 1100
↓
7 B 8 C
0111101110001100 = 7B8C

Binario a octal

011 101 100 110
↓
3 5 4 6
011101100110 = 3546

OPERACIONES ARITMÉTICAS CON NÚMEROS BINARIOS

Suma

Posibles
combinaciones

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

Equivale a "0 y me llevo 1"

$$\begin{array}{r} 1 \quad 11 \\ 10011100 \\ + 11001010 \\ \hline 101100110 \end{array}$$

Realiza las siguientes sumas:

$$\begin{array}{r} 101101 \\ + \underline{001011} \end{array}$$

$$\begin{array}{r} 100111 \\ + \underline{111000} \end{array}$$

$$\begin{array}{r} 110 \\ + \underline{100} \end{array}$$

Resta

Restas básicas

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

$$0 - 1 = 1$$

Equivale a "1 y me llevo 1"

Handwritten binary subtraction example showing borrowing:

$$\begin{array}{r} 1011101 \\ - 0001110 \\ \hline 1001111 \end{array}$$

Red arrows indicate borrowing from the 1011101 number to the 0001110 number, with the result 1001111 shown below the line.

Realiza las siguientes restas:

$$\begin{array}{r} 101101 \\ - 001011 \\ \hline \end{array}$$

$$\begin{array}{r} 100111011 \\ - 011100101 \\ \hline \end{array}$$

$$\begin{array}{r} 1010 \\ - 0101 \\ \hline \end{array}$$

Operaciones lógicas

49

Las operaciones lógicas se usan intensivamente en dispositivos electrónicos e informáticos. Las operaciones lógicas son **AND** (Y lógico), **OR** (O lógico), **NOT** (negación) y **XOR** (OR Exclusivo)

A	B	A AND B	A OR B	NOT B	A XOR B
0	0	0	0	1	0
0	1	0	1	0	1
1	0	0	1		1
1	1	1	1		0

Realiza las siguientes operaciones binarias:

01010101 AND 10111011 =

00101010 AND 11101101 =

00010100 OR 11010010 =

10001010 OR 10010100 =

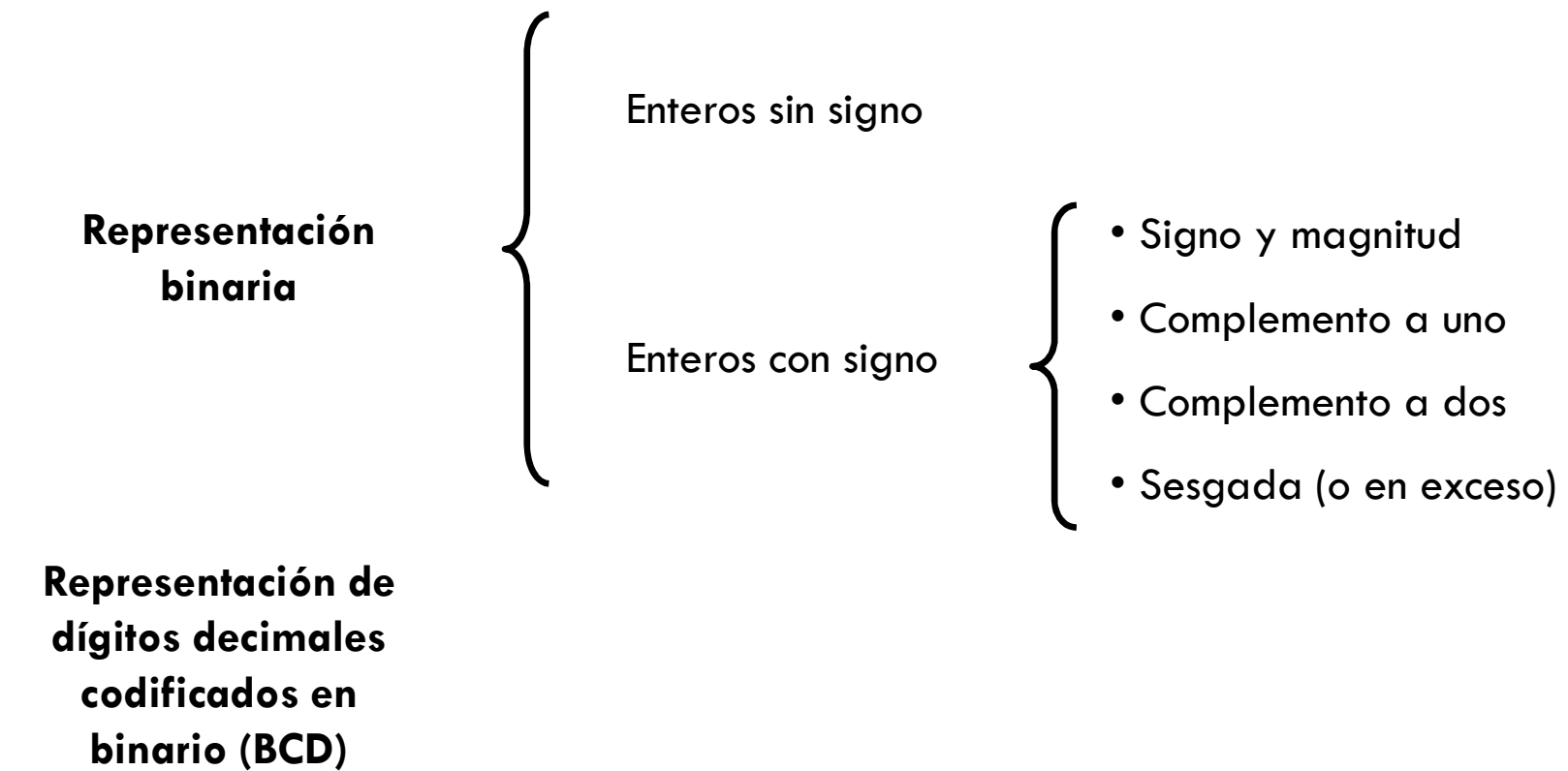
10010100 XOR 10100101 =

11011000 XOR 10010011 =

NOT 10010100 =

NOT 10100111 =

REPRESENTACIÓN DE DATOS ENTEROS



1 - Signo y magnitud

52

SIGNO - MAGNITUD



El signo se representa con el bit más significativo del dato.



Este bit es 0 si el número es positivo y 1 si es negativo.

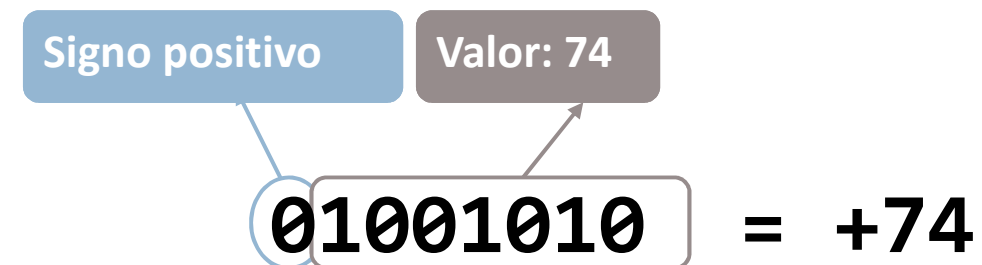
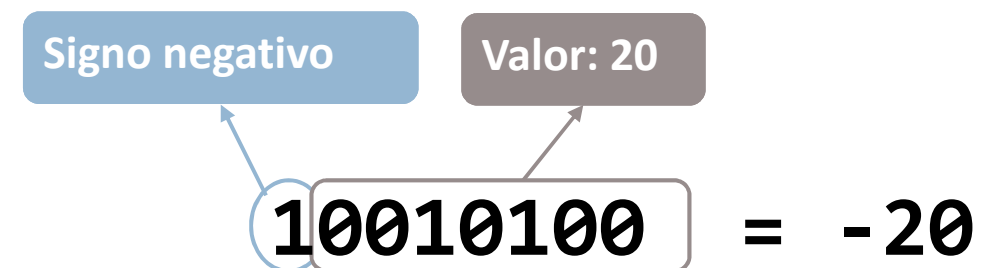


El resto de bits representan el valor del número en binario natural.



Valores límites: $-(2^{n-1}-1)$ y $2^{n-1}-1$ con $n = n^\circ$ de bits

1 - Signo y magnitud



- El principal inconveniente de este método es que hay dos representaciones para el número 0

Complementos

El método de **complementos** es una técnica que se utiliza para restar un número de otro utilizando sólo la suma de **números** positivos. Este método se utiliza comúnmente en las calculadoras mecánicas y todavía **se utiliza en** las computadoras modernas.

2 - Complemento a 1

55

Complemento a 1



El signo se representa mediante el bit más significativo



El resto de bits:

Si el número es positivo → valor absoluto del número en binario natural

Si el número es negativo → el complemento a uno del número.



El complemento a 1 se obtiene reemplazando ceros por unos y viceversa

2-Complemento a 1

Signo positivo

Valor: 74

Igual que signo y magnitud

$$\boxed{01001010} = +74$$

Signo negativo

Complemento a 1 de 1101011 (107)

$$\boxed{10010100} = -107$$

Signo negativo

Complemento a 1 de 10 (1010)

$$\boxed{10101} = -10$$

2-Complemento a 1

Inconveniente: 2 representaciones del número 0

Representaciones del "0": 00000000, 11111111

Ventaja: simplifica las operaciones aritméticas.

Se pueden restar dos números binarios sumando al minuendo el complemento a uno del sustraendo y sumando al resultado la cifra que se arrastra de dicho resultado.

$$\begin{array}{r}
 25 - 13 \\
 \hline
 25 = 011001 \\
 13 = 001101 \\
 -13 = 110010 \\
 \hline
 011001 \\
 + 110010 \\
 \hline
 1\ 001011 \\
 + 1 \\
 \hline
 001100 = 12
 \end{array}$$

3-Complemento a 2

Matemáticamente se define como

$$C_2^N = 2^n - N$$

siendo n el n° de bits del número N.

Se puede obtener fácilmente obteniendo el complemento a 1 y sumándole 1.

Ventajas:

Solo hay una representación para el número cero.

La resta es análoga al complemento a uno pero descartando el bit de acarreo.

3-Complemento a 2 - Ejemplo

$C2N = 2^n - N$ siendo n el nº de bits del número N

Ejemplo: El complemento a 2 del nº binario 1010, si tenemos 4 dígitos para la representación es:

$$C2(1010) = 2^4 - 1010 = 10000 - 1010 = \underline{0110}$$

Se puede obtener haciendo el complemento a 1 y sumándole 1:

$$C2 \text{ de } 1010 = C1 \text{ de } 1010 (0101) + 1 = \underline{0110}$$

4- Representación Exceso a M o sesgada

60

Una peculiaridad de los tres métodos de representación vistos es que si ordenamos todos los números de menor a mayor, las cifras binarias obtenidas no siguen un orden lexicográfico.

Ejemplo: -1 es menor que 1, pero si uso la representación “magnitud y signo con cuatro bits”, “1001” que representa a -1 no es menor que “0001” que representa a 1.

La representación en exceso a M consiste en sumarle al n° N el valor M (normalmente $M=2^{n-1}$ siendo n el n° de bits disponibles). De esta forma, los números negativos se convierten en positivos y no se necesita bit de signo.

Por ejemplo, utilizando 8 bits, tenemos las siguientes representaciones en exceso a 128:

Decimal	Exceso a 128	Binario exceso a 128
-128	$-128 + 128 = 0$	00000000
-127	$-127 + 128 = 1$	00000001
-1	$-1 + 128 = 127$	01111111
0	$0 + 128 = 128$	10000000
1	$1 + 128 = 129$	10000001
126	$126 + 128 = 254$	11111110
127	$127 + 128 = 255$	11111111

En la tabla anterior, se ve que los números y su representación siguen el mismo orden.

4- Representación Exceso a M o sesgada

62



Los números se tratan como si no tuvieran signo.



Se les “desplaza” de su valor por medio de la suma de otro número conocido como **exceso** o **desplazamiento (valor K)**.

Ejemplo: Exceso a 2^{n-1} . Si el dato es de 8 bits la cantidad que se debe sumar al número será $2^7=128$



Ventaja: No hay empaquetación del número. Por esto nos referimos a que no hay que recordar que partes del número son signo y significando, sino que los n -bits, **son** el número. Se utiliza principalmente en exponentes en representaciones de punto flotante.

	Tipo de representación			
Decimal	Signo y magnitud	Complemento a 1	Complemento a 2	Sesgado
7	0111	0111	0111	1111
6	0110	0110	0110	1110
5	0101	0101	0101	1101
4	0100	0100	0100	1100
3	0011	0011	0011	1011
2	0010	0010	0010	1010
1	0001	0001	0001	1001
0	0000	0000	0000	1000
0	1000	1111	---	---
-1	1001	1110	1111	0111
-2	1010	1101	1110	0110
-3	1011	1100	1101	0101
-4	1100	1011	1100	0100
-5	1101	1010	1011	0011
-6	1110	1001	1010	0010
-7	1111	1000	1001	0001
-8	---	---	1000	0000

Sesgado:
Se suma 8
(2^{4-1})

Ejemplos

64

Exceso a Z

Representar el número **-63** con 8 bits en exceso a $2^{n-1}=128$

▣ $-63+128=65$ La representación sería **01000001**

Si quisiera representar el número **63** en exceso a 128 :

▣ $63+128=191$ Su representación sería **10111111**

Practicamos...

65

EJERCICIOS

- Representa el número +93 en binario exceso 128 con 8 bits.
- Indica la representación decimal de 011010011 sabiendo que está representado en exceso 128 con 8 bits.

Representaciones BCD

66

Representar BCD
(Binary Coded
Decimal)

Cada dígito decimal se codifica aisladamente en 4 dígitos binarios

La codificación de cada dígito depende del tipo de BCD (BCD natural, BCD Aiken, exceso a 3, Gray...)

Inconveniente:

- Poco eficiente, ya que en 4 bits se pueden almacenar 16 valores diferentes y solo se almacenan 10.

Ventajas

- Simplifica la conversión entre binario y decimal.

BCD natural

67

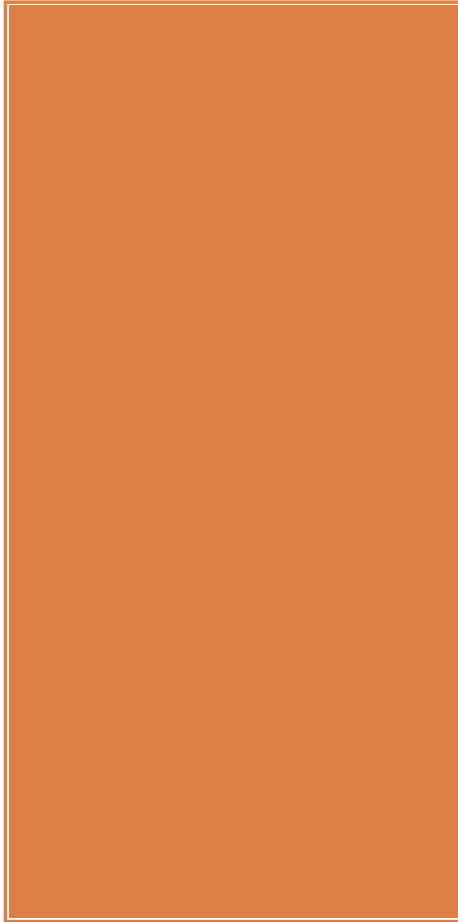
Dígito decimal	Código BCD
0	0000
1	0001
2	0010
3	0011
4	0100

Dígito decimal	Código BCD
5	0101
6	0110
7	0111
8	1000
9	1001

Las distintas variantes de BCD conceden pesos distintos a cada uno de los bits binarios, para facilitar la implementación de las operaciones aritméticas

Representación de números reales

68

- 
- Los humanos utilizamos fundamentalmente dos formas de representar los números reales: la notación tradicional o la científica. De cada una de ellas se deriva un tipo de representación utilizada en los ordenadores:
 - ▣ Coma fija
 - ▣ Coma flotante

Coma fija

69

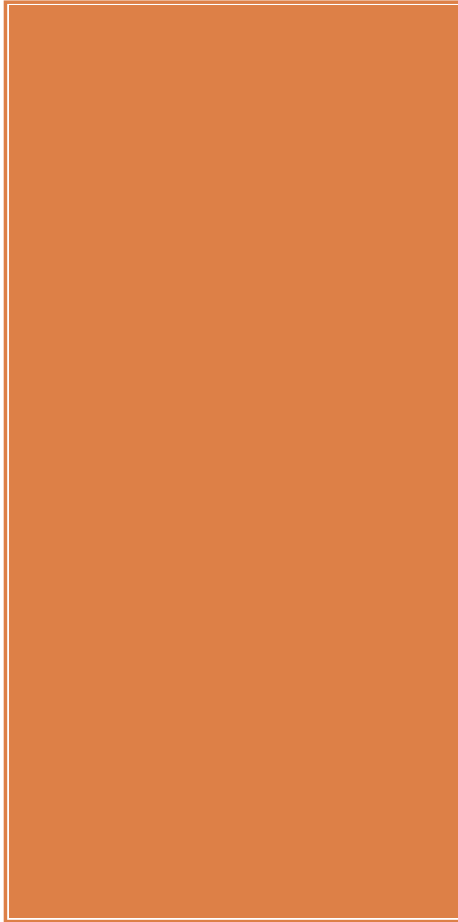
- Cuando representamos un real, utilizamos la coma para separar la parte entera de la real.
- La representación en coma fija utiliza la misma idea, por ejemplo, podemos usar 8 bits para representar un número, donde los 5 primeros bits se utilizan para la parte entera y los 3 restante para la parte real.

Ejemplo:

$$10101'110 = 1*2^4 + 1*2^2 + 1*2^0 + 1*2^{-1} + 1*2^{-2} = 21'75_{10}$$

Coma fija

70

- 
- Ventaja: se pueden utilizar los mismos circuitos hardware para implementar las operaciones que se utilizan con la representación entera.
 - Inconveniente: tiene limitada la precisión y ésta es fija para todos los valores. (Si tienes 5 bits para enteros y 3 para la parte real no se podrán representar números mayores o iguales a 32 (2^5), ni números con más precisión de 0,125 (2^{-3}))

Coma flotante

- Cuando se opera con números muy grandes o muy pequeños se suele utilizar la **notación exponencial (notación científica o notación en coma flotante)**

- Cada número se puede expresar como:

$$N = M \cdot B^E$$

Por ejemplo:

$$\begin{aligned} 13.257,3285 &= 13.257,3285 \cdot 10^0 = \\ 1,32573285 \cdot 10^4 &= 132.573.285 \cdot 10^{-4} = \dots \end{aligned}$$

- Se llama de coma flotante debido a que la coma decimal no se halla en una posición fija dentro de la secuencia de bits, sino que su posición se indica como una potencia de la base.
- Estos números tienen tres partes:



- Dado que un número se puede representar de diferentes maneras es habitual representar los números en punto flotante de forma **normalizada**.
- La normalización consiste en ajustar la coma decimal de la mantisa de forma que quede a la derecha del dígito no nulo más significativo.

IEEE 754

- Es un estándar para la representación de número en punto flotante.
- Se contemplan dos formatos:
 - ▣ Simple precisión: 32 bits

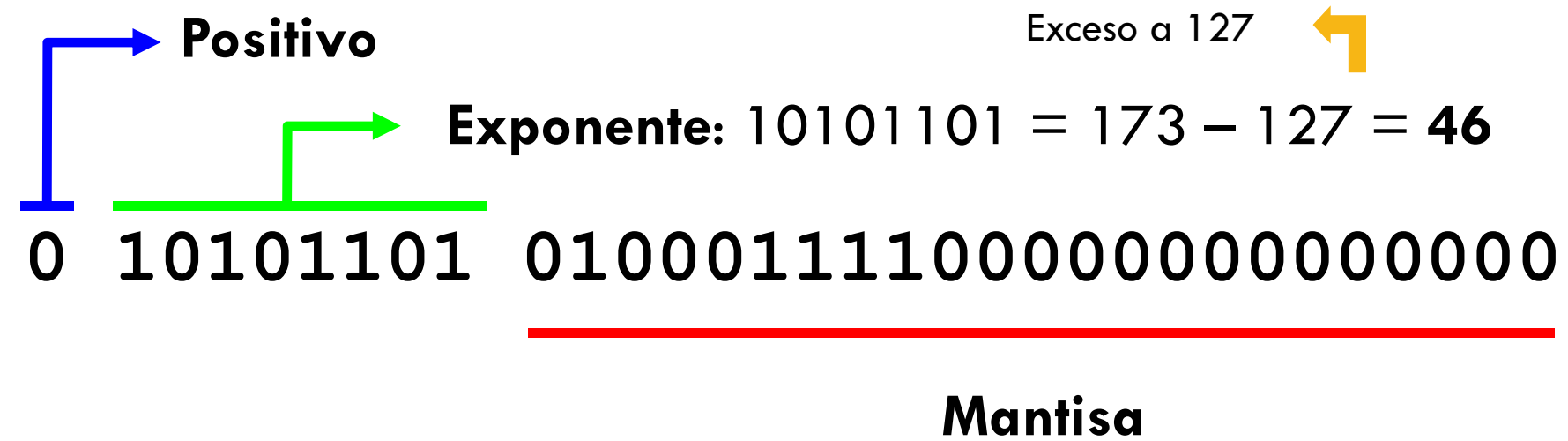
1 bit	8 bits	23 bits
Signo	Exponente	Mantisa

- ▣ Doble precisión: 64 bits

1 bit	11 bits	52 bits
Signo	Exponente	Mantisa

□ Características:

- El signo es 0 si positivo y 1 si negativo
- El exponente se representa en exceso a 127 para simple precisión y exceso 1023 para doble precisión.
- La mantisa se representa en binario natural.
- La coma está a la derecha del primer bit significativo.



EJEMPLO – Convertir 163'75 a coma flotante en precisión simple

77

Lo haremos en tres pasos:

- 1- Pasar el número de decimal a binario
- 2- Ponemos el nº binario en notación científica
- 3- Pasamos de notación científica al estándar IEEE754
(notación simple: 1 bit signo + 8 bits exponente en exceso
127 + 23 bits para la mantisa)

EJEMPLO – Convertir 163'75 a coma flotante en precisión simple

78

163,75

① De decimal a binario

163 $\div 2$
 03 81 $\div 2$
 1 01 40 $\div 2$
 0 20 $\div 2$
 0 10 $\div 2$
 0 5 $\div 2$
 1 2 $\div 2$
 1 1

0,75 Multiplicar por 2 hasta llegar a 1

0,75 $\times 2$ 1,50
 0,50 $\times 2$ 1,00

1010001111

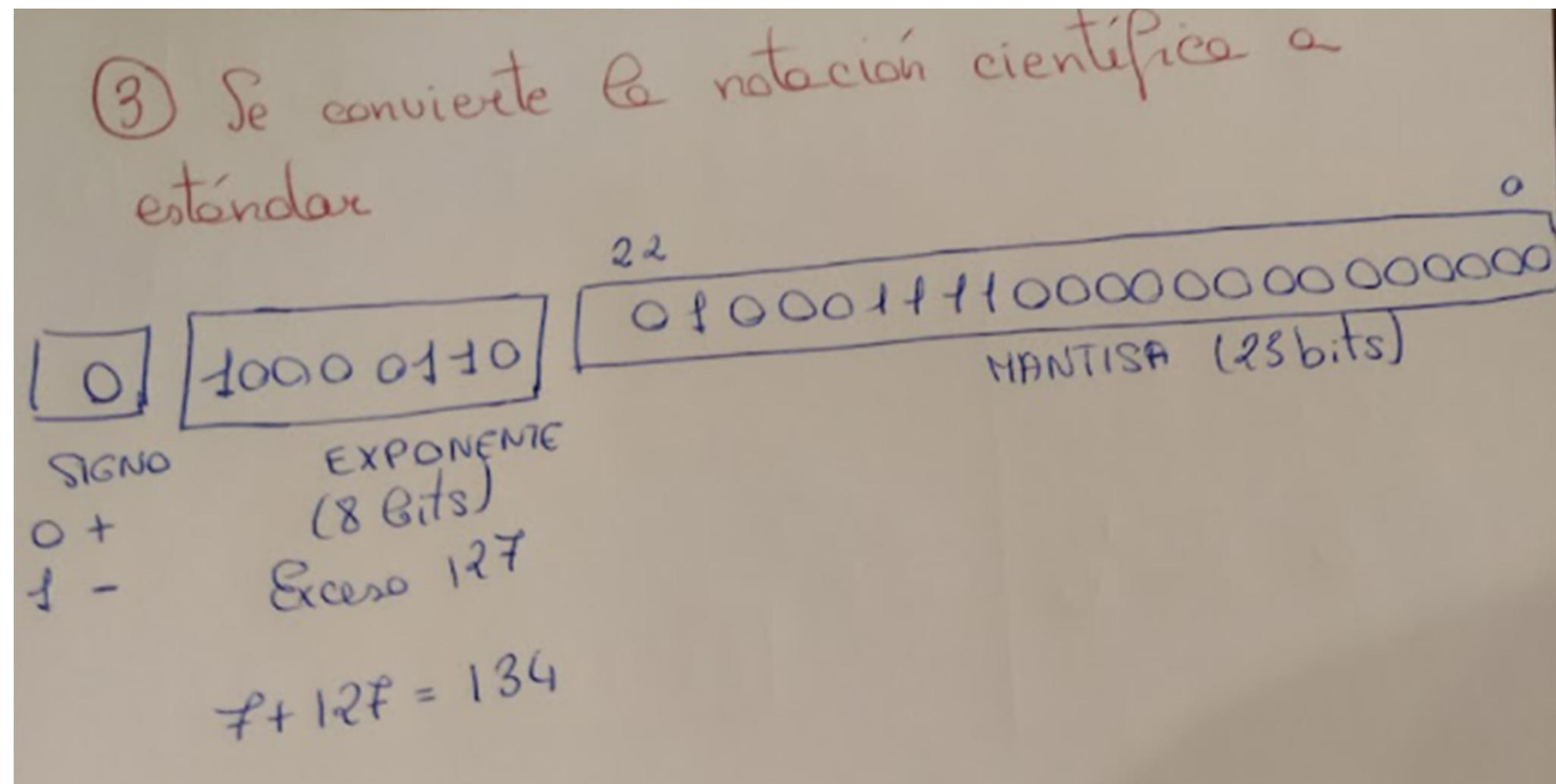
② Ponemos el binario en notación científica

1010001111 $\times 2^7$

(Si hubiera sido 0,001111 $\Rightarrow 1111 \times 2^{-3}$)

EJEMPLO – Convertir 163'75 a coma flotante en precisión simple

79



EJERCICIOS

80

1. Convertir el número binario $-101111110,011$ a coma flotante con precisión simple.
2. Convertir el número $-284,625_{10}$ a coma flotante en precisión simple.

La CPU solo es capaz de manejar números, por lo que el resto de los datos se deben representar como símbolos numéricos.

6

SISTEMAS DE CODIFICACIÓN ALFANUMÉRICA

Se puede pensar en los diferentes tipos de representaciones como tablas con correspondencias entre los valores numéricos representados por los bits y los caracteres a los que representan.

6.1.- CÓDIGO ASCII

ASCII			ASCII			ASCII			ASCII		
Dec Hex		Simbolo	Dec Hex		Simbolo	Dec Hex		Simbolo	Dec Hex		Simbolo
0	0	NUL	16	10	DLE	32	20	(space)	48	30	0
1	1	SOH	17	11	DC1	33	21	!	49	31	1
2	2	STX	18	12	DC2	34	22	"	50	32	2
3	3	ETX	19	13	DC3	35	23	#	51	33	3
4	4	EOT	20	14	DC4	36	24	\$	52	34	4
5	5	ENQ	21	15	NAK	37	25	%	53	35	5
6	6	ACK	22	16	SYN	38	26	&	54	36	6
7	7	BEL	23	17	ETB	39	27	'	55	37	7
8	8	BS	24	18	CAN	40	28	(	56	38	8
9	9	TAB	25	19	EM	41	29	)	57	39	9
10	A	LF	26	1A	SUB	42	2A	*	58	3A	:
11	B	VT	27	1B	ESC	43	2B	+	59	3B	;
12	C	FF	28	1C	FS	44	2C	,	60	3C	<
13	D	CR	29	1D	GS	45	2D	-	61	3D	=
14	E	SO	30	1E	RS	46	2E	.	62	3E	>
15	F	SI	31	1F	US	47	2F	/	63	3F	?

6.1.- CÓDIGO ASCII

ASCII			ASCII			ASCII			ASCII		
Dec Hex		Símbolo	Dec Hex		Símbolo	Dec Hex		Símbolo	Dec Hex		Símbolo
64	40	@	80	50	P	96	60	`	112	70	p
65	41	A	81	51	Q	97	61	a	113	71	q
66	42	B	82	52	R	98	62	b	114	72	r
67	43	C	83	53	S	99	63	c	115	73	s
68	44	D	84	54	T	100	64	d	116	74	t
69	45	E	85	55	U	101	65	e	117	75	u
70	46	F	86	56	V	102	66	f	118	76	v
71	47	G	87	57	W	103	67	g	119	77	w
72	48	H	88	58	X	104	68	h	120	78	x
73	49	I	89	59	Y	105	69	i	121	79	y
74	4A	J	90	5A	Z	106	6A	j	122	7A	z
75	4B	K	91	5B	[	107	6B	k	123	7B	{
76	4C	L	92	5C	\	108	6C	l	124	7C	
77	4D	M	93	5D	]	109	6D	m	125	7D	}
78	4E	N	94	5E	^	110	6E	n	126	7E	~
79	4F	O	95	5F	_	111	6F	o	127	7F	

Ejemplos (código ASCII en hexadecimal):

- 61_{ASCII} → a
- 41_{ASCII} → A
- arbol → 6172626F6C_{ASCII}
- Hola! → 486F6C6121_{ASCII}
- 432E492E462E502E_{ASCII} → C.I.F.P.

6.- SISTEMAS DE CODIFICACIÓN ALFANUMÉRICA

84

- ASCII (American Standard Code for Information Interchange) (ISO-646-IRV): 7 bits (128 símbolos), el más extendido, pensado para el Inglés
- ISO-8859-1 (Latin 1): Extensión del ASCII, 8 bits (256 símbolos), incluye las lenguas de Europa occidental y otras.
- ISO-8859-15 (Latin 9): Modificación del Latin 1 para incorporar el símbolo del EURO y otras actualizaciones.
- ISO-8859-2 a ISO-8859-14: Extensiones del ASCII para diferentes lenguas: cirílico, árabe, griego, hebreo, etc.
- ISO-10646 (Unicode): código de 16 bits que engloba a todas las codificaciones e idiomas conocidos.

2.5.3.- UNICODE

85

Juegos de caracteres **ISO-8859**

DESVENTAJA

Diferentes para cada idioma

Para **leer** cualquier documento codificado con ellos hay que **conocer el sistema que usa**

SOLUCIÓN

Desarrollar un **único código de caracteres universal**

Contenidos **todos los símbolos existentes en el mundo**

UNICODE

<http://unicode-table.com/es/>

2.5.3.- UNICODE

86

PROBLEMAS

Necesita **varios bytes** (2 bytes para la versión 3.0 o 3 para las posteriores) **para cada carácter** en vez de uno solo

Si abrimos un documento con caracteres, aunque sean simples, nos **aparecen símbolos “extraños”** de por medio.

SOLUCIÓN



UTF-8

Unicode **T**ransformation **F**ormat

2.5.3.- UTF-8

87

UTF-8

Unicode Transformation Format

Es el sistema de codificación que se está imponiendo
actualmente para representar juegos de caracteres

Longitud de carácter **variable**

Los **caracteres más comunes** (ASCII)
los codifica con **1 byte**

Los **caracteres menos frecuentes** utilizan **2, 3, 4...**
bytes (cuanto menos frecuentes, más bytes utilizan)

[¿Cómo convertir de Unicode a UTF-8? - YouTube](#)

2.5.3.- UTF

Usa una estructura variable en la cual el **número de 1s al principio del primer byte** indica **el número total de bytes del carácter**

Rango Unicode (hexadecimal)	UTF-8 secuencia de octetos (binario)
-----+-----	
0000 0000-0000 007F	0xxxxxxx
0000 0080-0000 07FF	110xxxxx 10xxxxxx
0000 0800-0000 FFFF	1110xxxx 10xxxxxx 10xxxxxx
0001 0000-0010 FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
0020 0000-03FF FFFF	111110xx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx
0400 0000-7FFF FFFF	1111110x 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx 10xxxxxx

Esta tabla indica el **número de bytes** y **estructura** a usar en función del rango en el que está el **Punto de Código** a representar

El espacio marcado con **xxx...** debe de ser sustituido por el **punto de código** del carácter a representar en binario, rellenando con 0s a la izquierda si hiciera falta

2.5.3.- UTF

89

- Pasar el carácter ß → ?_{UTF-8}

1.- Buscando en **tablas de Unicode** vemos que ß tiene el **punto de código DF** (U+00DF)

2.- Buscando en la tabla, vemos que **DF** está en el **rango 80-7FF**, por lo que le corresponde un formato **110xxxxx 10xxxxxx**.

0000 0080-0000 07FF | 110xxxxx 10xxxxxx

3.- **DF** en binario es **1101 1111**

4.- Colocamos los bits de **DF** en el lugar de las x rellenando con 0 a la izquierda

110**00011** 10**011111** = C 3 9 F



2.5.3.- UTF

90

Ejemplos:

- $C394_{UTF-8} \rightarrow \hat{O}$
- $35E282ACC5BD_{UTF-8} \rightarrow 5\text{€}\check{Z}$
- $\text{árbol} \rightarrow C3A172626F6C_{UTF-8}$
- $\text{¡Hola!} \rightarrow C2A1486F6C6121_{UTF-8}$
- $432E492E462E502E_{UTF-8} \rightarrow \text{C.I.F.P.}$

EJERCICIOS

91

1. Convertir a UTF8 los siguientes caracteres:

1. H
2. o
3. L
4. a

2.5.- SISTEMAS DE CODIFICACIÓN ALFANUMÉRICA

92

Estos **sistemas de codificación** (vistos hasta ahora)

Son utilizados por

Programas **EDITORES** de texto, que guardan únicamente texto, sin formato (**.TXT**)

Los programas **PROCESADORES** de texto han de guardar también **información extra**

márgenes, fuentes, tamaños, subrayados, colores, alineados...

Añadir códigos adicionales al texto en formato ISO-8859-15 o UTF-8 (**.DOC, .ODT**)

UNIDADES DE ALMACENAMIENTO DE DATOS

93

Unidades de Medidas de Almacenamiento

Medida	Simbologia	Equivalencia	Equivalente en Bytes
byte	b	8 bits	1 byte
kilobyte	Kb	1024 bytes	1 024 bytes
megabyte	MB	1024 KB	1 048 576 bytes
gigabyte	GB	1024 MB	1 073 741 824 bytes
terabyte	TB	1024 GB	1 099 511 627 776 bytes
Petabyte	PB	1024 TB	1 125 899 906 842 624 bytes
Exabyte	EB	1024 PB	1 152 921 504 606 846 976 bytes
Zetabyte	ZB	1024 EB	1 180 591 620 717 411 303 424 bytes
Yottabyte	YB	1024 ZB	1 208 925 819 614 629 174 706 176 bytes
Brontobyte	BB	1024 YB	1 237 940 039 285 380 274 899 124 224 bytes
Geopbyte	GB	1024 BB	1 267 650 600 228 229 401 496 703 205 376 bytes

EJERCICIOS RESUELTOS

94

1. $1024 \text{ bits} = 1024/8 = 128 \text{ B}$

2. $2 \text{ MB} = 2 * 1024 * 1024 = 2.097.152 \text{ B}$

3. $2048 \text{ MB} = 2048 / 1024 = 2 \text{ GB}$

EJERCICIOS

95

A resolver...

1. ¿Cuántos KB hay en 3 MB?
2. ¿Cuántos MB hay en 5 GB?
3. ¿Cuántos GB hay en 20 TB?
4. ¿Cuántos TB hay en 5000 GB?
5. ¿Cuántos TB hay en 20000 GB?
6. ¿Cuántos TB hay en 100000 MB?
7. ¿Cuántos GB hay en 30000 MB?
8. ¿Cuántos MB hay en 12000 TB?
9. ¿Cuántos KB hay en 2 TB?
10. ¿Cuántos Bits hay en 2 Byte?
11. ¿Cuántos Bits hay en 1 KB?
12. ¿Cuántos MB hay en 15 TB?
13. ¿Cuántos KB hay en 2 TB?
14. ¿Cuántos MB hay en 18 GB?
15. ¿Cuántos KB hay en 35 GB?
16. ¿Cuántos MB hay en 35000 KB?
17. ¿Cuántos TB hay en 1800000 KB?
18. ¿Cuántos MB hay en 34 GB?

Prefijos del SI			
Prefijo	Símbolo	Factor	Equivalencia decimal
yotta	Y	10 ²⁴	1 000 000 000 000 000 000 000 000
zetta	Z	10 ²¹	1 000 000 000 000 000 000 000 000
exa	E	10 ¹⁸	1 000 000 000 000 000 000 000
peta	P	10 ¹⁵	1 000 000 000 000 000 000
tera	T	10 ¹²	1 000 000 000 000 000
giga	G	10 ⁹	1 000 000 000
mega	M	10 ⁶	1 000 000
kilo	k	10 ³	1 000
hecto	h	10 ²	100
deca	da	10 ¹	10
sin prefijo		1	1
deci	d	10 ⁻¹	0.1
centi	c	10 ⁻²	0.01
mili	m	10 ⁻³	0.001
micro	μ	10 ⁻⁶	0.000 001
nano	n	10 ⁻⁹	0.000 000 001
pico	p	10 ⁻¹²	0.000 000 000 001
femto	f	10 ⁻¹⁵	0.000 000 000 000 001
atto	a	10 ⁻¹⁸	0.000 000 000 000 000 001
zepto	z	10 ⁻²¹	0.000 000 000 000 000 000 001
yocto	y	10 ⁻²⁴	0.000 000 000 000 000 000 000 001

Durante mucho tiempo se han utilizado los prefijos kilo-, mega, giga-, tera-, etc. como potencias de 2¹⁰, 2²⁰, 2³⁰, y así sucesivamente, pero se crearon los prefijos binarios para diferenciarlos de los prefijos del sistema decimal, y así surgieron los prefijos kibi-, mebi-, gibi-, tebi-, etc.

Binario			Decimal y diferencia con binario			
Símbolo	Prefijo	Factor	Factor	Prefijo	Bin÷Dec	Error
Ki	Kibi	2 ¹⁰	10 ³	Kilo	1.024	2.4%
Mi	Mebi	2 ²⁰	10 ⁶	Mega	1.049	4.9%
Gi	Gibi	2 ³⁰	10 ⁹	Giga	1.074	7.4%
Ti	Tebi	2 ⁴⁰	10 ¹²	Tera	1.100	10.0%
Pi	Pebi	2 ⁵⁰	10 ¹⁵	Peta	1.126	12.6%
Ei	Exbi	2 ⁶⁰	10 ¹⁸	Exa	1.153	15.3%
Zi	Zebi	2 ⁷⁰	10 ²¹	Zetta	1.181	18.1%
Yi	Yobi	2 ⁸⁰	10 ²⁴	Yotta	1.209	20.9%

Medidas de frecuencia

Múltiplos del Sistema Internacional para hercio (Hz)

Submúltiplos			Múltiplos		
Valor	Símbolo	Nombre	Valor	Símbolo	Nombre
10 ⁻¹ Hz	dHz	decihercio	10 ¹ Hz	daHz	decahercio
10 ⁻² Hz	cHz	centihercio	10 ² Hz	hHz	hectohercio
10 ⁻³ Hz	mHz	milihercio	10 ³ Hz	kHz	kilohercio
10 ⁻⁶ Hz	μHz	microhercio	10 ⁶ Hz	MHz	megahercio
10 ⁻⁹ Hz	nHz	nanohercio	10 ⁹ Hz	GHz	gigahercio
10 ⁻¹² Hz	pHz	picohercio	10 ¹² Hz	THz	terahercio
10 ⁻¹⁵ Hz	fHz	femtohercio	10 ¹⁵ Hz	PHz	petahercio
10 ⁻¹⁸ Hz	aHz	attohercio	10 ¹⁸ Hz	EHz	exahercio
10 ⁻²¹ Hz	zHz	zeptohercio	10 ²¹ Hz	ZHz	zettahercio
10 ⁻²⁴ Hz	yHz	yoctohercio	10 ²⁴ Hz	YHz	yottahercio
Los prefijos más comunes están en negrita.					

El reloj del microprocesador genera una serie de pulsos de forma constante y periódica que hacen que el resto del sistema funcione de manera sincronizada. Cuanto más rápido emita esas señales, se dirá que trabaja a mayor frecuencia. La frecuencia se mide en hercios. Es una medida inversa al tiempo.

Medidas de rendimiento

98

La unidad de medida FLOPS (Floating Point Operations per Second, operaciones de coma flotante por segundo), se utiliza para medir el rendimiento en grandes ordenadores y en tarjetas gráficas, midiendo el número de operaciones en coma flotante por segundo.

■ Rendimiento computacional en nº de operaciones de coma flotante por segundo (flops)

NOMBRE	UNIDAD	ESCALA
kiloflops	Kflops	10^3
megaflops	Mflops	10^6
gigaflops	Gflops	10^9
teraflops	Tflops	10^{12}
petaflops	Pflops	10^{15}
exaflops	Eflops	10^{18}

Medidas de transferencia de información

99

Entre las medidas de transferencia de la información se utiliza la unidad bits por segundo (bps) que es la cantidad de bits que se puede transmitir en un segundo. Se suele llamar tasa de transferencia y mide la velocidad de transferencia en una conexión de red, en una unidad de disco, una memoria o entre un periférico y el ordenador.

Unidad de ancho de banda	Abreviatura	Equivalencia
Bits por segundo	bps	1 bps = unidad fundamental de ancho de banda
Kilobits por segundo	kbps	1 kbps = 1000 bps = 10^3 bps
Megabits por segundo	Mbps	1 Mbps = 1 000 000 bps = 10^6 bps
Gigabits por segundo	Gbps	1 Gbps = 1 000 000 000 bps = 10^9 bps
Terabits por segundo	Tbps	1 Tbps = 1 000 000 000 000 bps = 10^{12} bps

Medidas de electricidad

100

Vatio (W): Mide la potencia. Relacionado con la fuente de alimentación.

Voltio (V): Tensión o corriente eléctrica. Las fuentes de alimentación trabajan con salidas de 12V, 5V y 3,3V.

Amperio (A): Intensidad. Un amperio hora (Ah) o miliamperio hora (mAh) se utiliza para calcular la energía que se suministra en una hora. Relacionado con baterías.

Ohmio (Ohm, Ω): resistencia eléctrica.

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

101



Ya sabemos cómo representar **números** y **letras...**



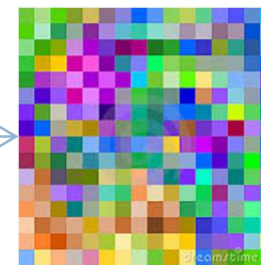
¿Representar imágenes y gráficos?

Gran parte de la **información** que manejamos hoy en un **ordenador** es **gráfica**



Exige una **gran capacidad de memoria**

Para codificar una imagen...



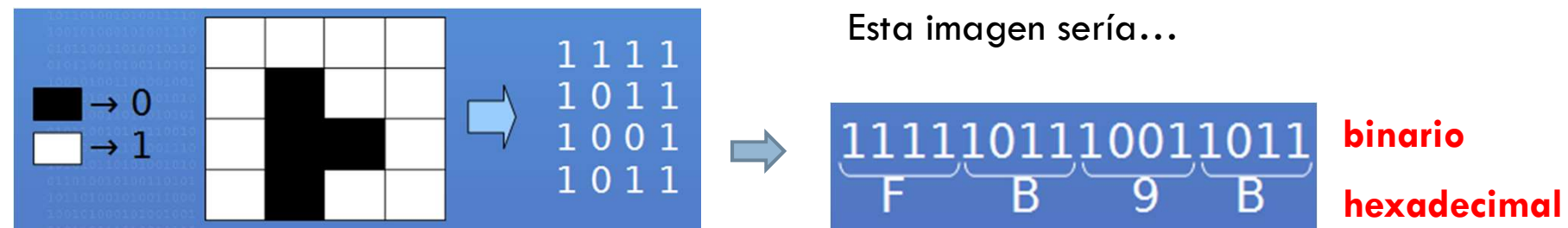
el **color** de cada
pixel se puede
representar con
uno o **varios bits**

Trama de numerosos **puntos**
de colores llamados **píxeles**

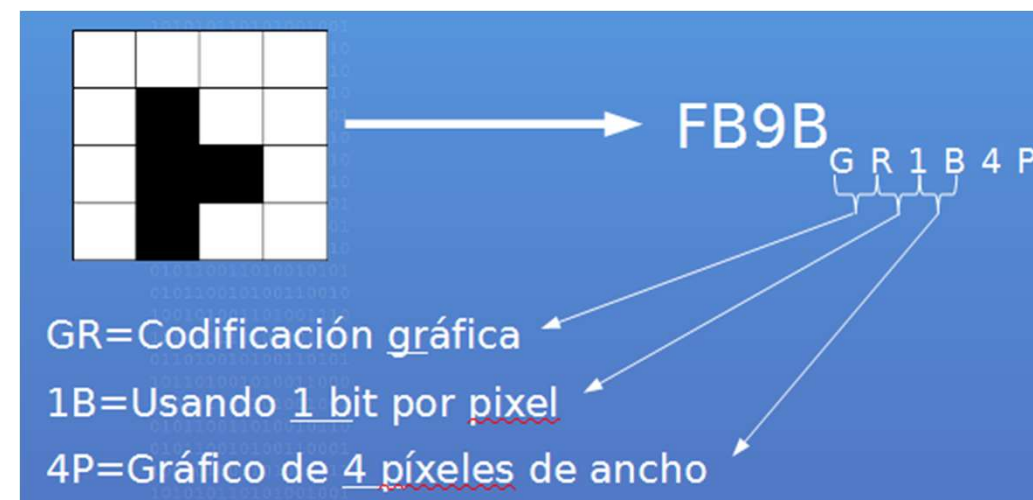
2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

102

Ejemplo: representar esta pieza del tetris como un gráfico de 1 bpp (bit por pixel)



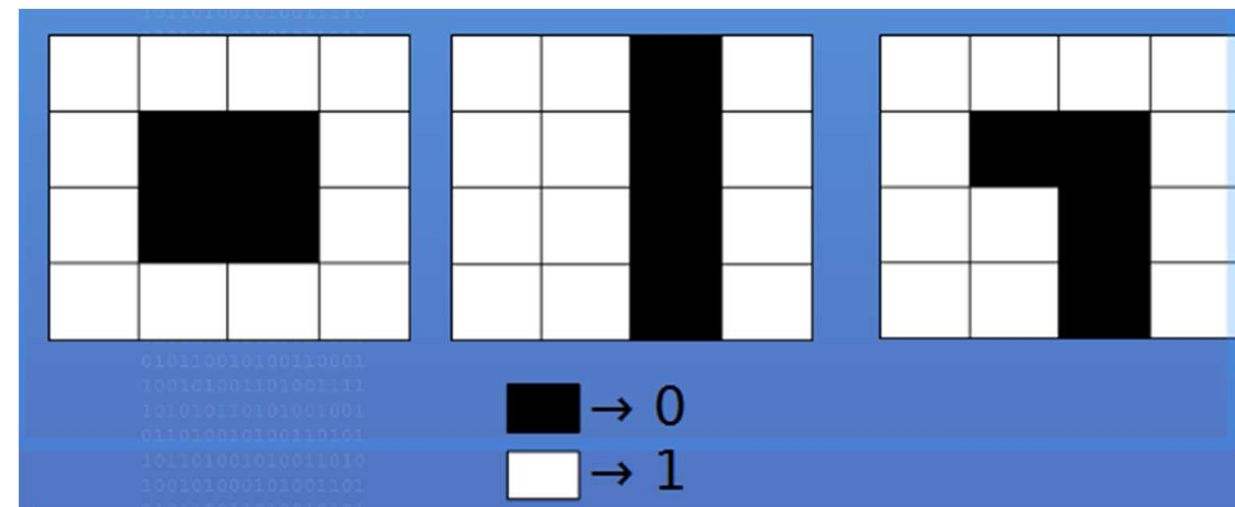
Por lo tanto podríamos decir que:



2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

103

Usando también **1 bit por píxel** y **4 bits de ancho**, codifica el resto de las piezas del Tetris:



¿...?

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

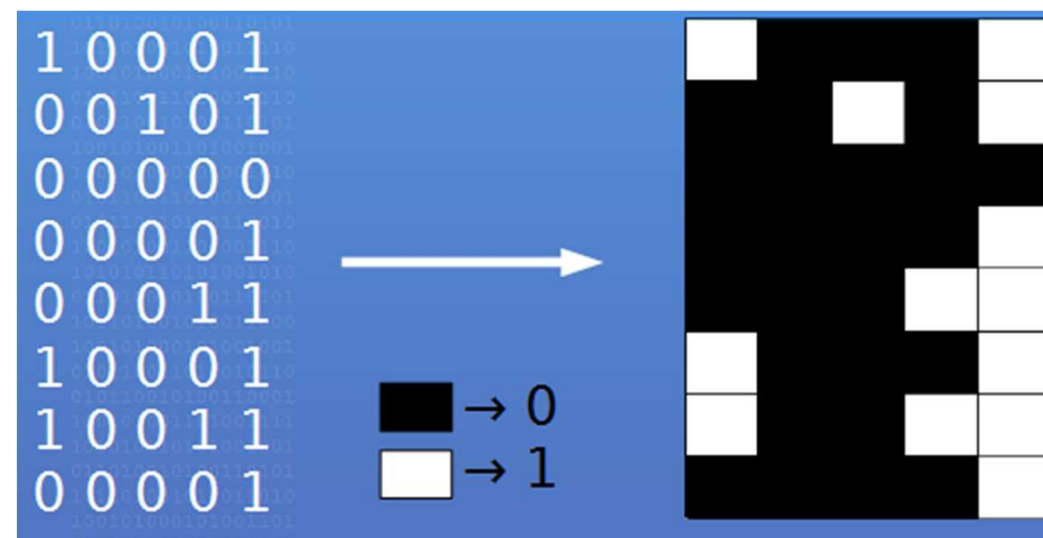
104

También podemos hacer lo contrario... $\longrightarrow$ **894011C661**_{GR1B5P} $\rightarrow$?

1.- Pasamos a binario

894011C661_{GR1B5P} $\rightarrow$ **100010010100000000100011100011001100001**₂

2.- Codificamos en bloques de 5 bits de ancho



2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

105

Decodifica como imagen de **1 bit por píxel** y **11 píxeles de ancho** lo siguiente:

208220FE376FFF7F6828D8_{GR1B11P} → ?

¿...?

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

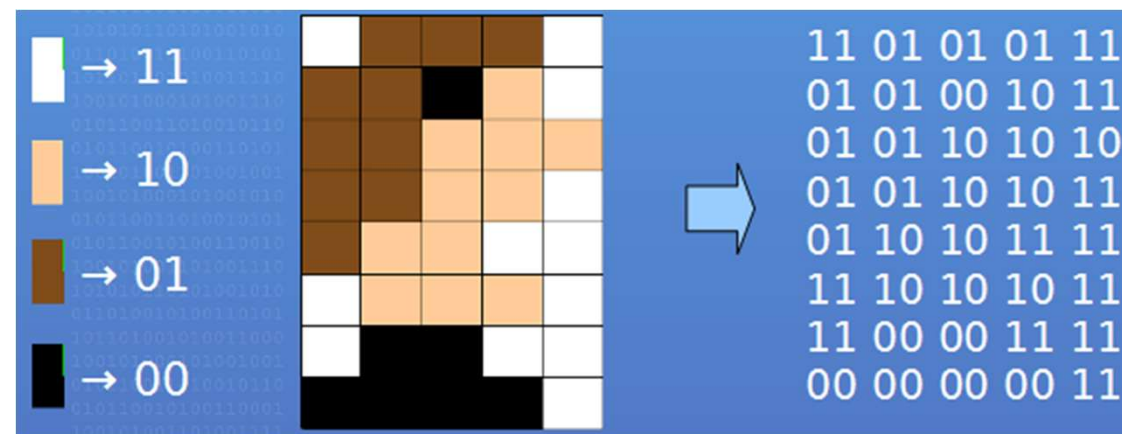
106

1 bit por pixel $\rightarrow 2^1 \rightarrow 2$ colores distintos



Si empleamos **más bits por píxel** $\rightarrow$ la imagen tendrá **tantos colores** como **combinaciones** se puedan hacer con esos bits

2 bit por pixel $\rightarrow 2^2 \rightarrow 4$ colores distintos



1101010111010100101101011010100101101011011010111111010101111000011110000000011
 D 5 D 4 B 5 A 9 6 B 6 B F A B C 3 C 0 3

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

107

Ahora haz lo contrario, decodifica como imagen de **2 bits por píxel** y **10 píxeles**...

003000ABA815AB47765555DD5DD55155741DD54055D001D400050000000_{GR2B10P} → ?

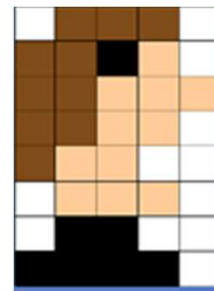


¿...?

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

108

La imagen anterior (**cara**) de 5x8 píxeles con 2 bpp (bits por pixel) es equivalente a:



D5D4B5A96B6BFABC3C03
G R 2 B 5 P

¿Cuánto ocupa? → $2 \text{ bits/pixel} \times 5 \text{ píxeles de ancho} \times 8 \text{ píxeles de alto} =$
 $80 \text{ bits} = 80 / 8 \text{ bytes} = 10 \text{ bytes}$



De esta manera podemos calcular **muy fácilmente** los **bits que ocupa una imagen**

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

109

Ejemplo: Calcular cuánto ocuparía una imagen de **320x200 píxeles** con **8 bits por píxel...**

$$8 \times 320 \times 200 = 512000 \text{ bits} = 512000/8 \text{ bytes} = 64000 \text{ bytes} \\ = 64000/1024 \text{ Kbytes} = 62,5 \text{ KiB}$$

Para resolver...

- ¿Cuanto ocuparía una imagen de **1024 bits de alto, 768 de ancho y 12 bpp?**
- ¿Y una imagen de **800x600 y 18 bpp?**
- ¿Y una foto de **3664 x 2748 y 36 bpp** (cámara Réflex a alta calidad)?

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

110

- También conviene saber... → ¿cuántos colores tiene una imagen con X bits?

Ejemplo: → ¿Cuántos **colores** tiene una imagen de 8 bits por píxel?

$$\text{Nº colores} = \text{nº combinaciones posibles} = 2^8 = 256 \text{ colores}$$

- Y conviene saber lo contrario... → ¿cuántos bits por píxel hacen falta para representar X colores?

Ejemplo: → ¿Cuántos **bits por píxel** hacen falta para representar 5000 colores en una imagen?

$$2^1 = 2 / 2^2 = 4, / 2^3 = 8 / 2^4 = 16 / 2^5 = 32 / 2^6 = 64 / 2^7 = 128 / 2^8 = 256 / 2^9 = 512 / 2^{10} = 1024 / 2^{11} = 2048 / 2^{12} = 4096 / 2^{13} = 8192 \rightarrow \text{Hacen falta por lo menos } 13 \text{ bits}$$

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

111

Para resolver...

- ¿Cuántos colores tiene una imagen de **16 bits por píxel**?
- ¿Cuántos colores tiene una imagen de **24 bits por píxel**?
- ¿Cuántos bits por píxel hacen falta para representar **dos millones** de colores?
- ¿Cuántos bits por píxel hacen falta para representar **12 colores**?

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

112

Para pensar...

¿Cuanto ocupa una imagen normal de fondo de escritorio que tiene **1280 x 1024 píxeles** y **16 millones de colores**?

¿...?

¿ Y una foto de alta calidad de **3600 x 2400** pixeles con **1000 millones de colores**?

¿...?

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

113

Para pensar...

¿Cuánto ocupa una imagen normal de fondo de escritorio que tiene **1280 x 1024 píxeles** y **16 millones de colores**?

Como $2^{24} = 16.777.216$ millones $\rightarrow$ **24 bits**

$1280 \times 1024 \times 24 = 31.457.280$ bits = $3.932.160$ bytes = **3,75 MiB**

¿ Y una foto de alta calidad de **3600 x 2400** pixeles con **1000 millones de colores**?

$2^{29} = 536.870.912$ / $2^{30} = 1.073.741.824 \rightarrow$ **30 bits**

$3600 \times 2400 \times 30 = 259.200.000$ bits = $32.400.000$ bytes = **30,9 MiB**

2.6.- SISTEMAS DE CODIFICACIÓN GRÁFICA

114

FORMAS DE ALMACENAR UNA IMAGEN

En bruto o raw

«Tal cual», **sin ningún tipo de compresión ni pérdida de calidad.**

Todos los tamaños que hemos calculado se refieren a este tipo (**.RAW**)

Sin pérdida (de calidad) o lossless

Se **comprime la información**, normalmente aprovechando patrones repetidos, sin perder ningún dato después de la descompresión (**.PNG**)

Con pérdida (de calidad) o lossy

Se **comprime la información** en base a generar una imagen no idéntica, en la que se pierden algunos detalles (**.JPG**)

Compresión de imagen